

23. | DeepGEMM

Chapter Abstract. Transformer projections share the algebra $D = C + AB^T$, but that equation does not select a production kernel. DeepGEMM must also resolve shape, payload dtype, scale layout, grouping, architecture, and just-in-time (JIT) state while preserving the declared output relation. Starting from a 6×8 by 12×8 multiplication, the running example carries a BF16 reference into low-precision and grouped-expert paths, then separates first-call compilation from warm execution. The selected path and its numerical check determine how the resulting timing can be interpreted. Later fused and adjacent primitives retain their own outputs and oracles rather than borrowing the GEMM result.

The equation $D = C + AB^T$, the BF16 reference, low-precision scale reconstruction, grouped-row semantics, and numerical tolerance define the accepted result. Tiling, staging, and JIT selection may change how the result is computed, but every declared K -slice must contribute before the output tile is stored and checked.

Hyperconnection pre-norm, finite einsum, and MQA/indexer-logit kernels are transfer cases with separate output relations. Mega MoE uses the GEMM and grouped-expert foundations but widens the execution boundary to multi-rank dispatch, two expert linears, activation, and combine; that fused systems path is taught in Chapter 27.

23.1. Why GEMM Is the Compute Primitive Under the Transformer

Later chapters reuse the same output relation while changing shape, dtype, scale layout, and launch path.

Attention and MLP projections, output logits, MoE experts, and training gradients all reduce to batches of matrix products. Their GPU calls nevertheless differ in what the row axis means, how values are represented, and whether rows are dense or routed. DeepGEMM resolves those differences into a legal low-precision kernel while preserving the declared output relation and acceptance tolerance [81]. The small dense case below fixes the algebra before later sections change its representation and execution path.

Definition 23.1.1 (GEMM) *A GEMM instance here is a matrix multiplication with optional accumulation. In the default orientation, $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{N \times K}$, $C, D \in \mathbb{R}^{M \times N}$, and the logical operation is*

$$D = C + AB^T.$$

When no accumulation matrix is supplied, the same definition is read with $C = 0$. The shape rule is that the two operands share the reduction dimension K , and the output has one row for each row of A and one column for each row of B .

The running dense example uses $M = 6$, $K = 8$, and $N = 12$. Thus A is a 6×8 activation matrix and B is a 12×8 weight matrix. In the local NT convention, N marks A as supplied and T marks the transpose of B in AB^T . The output D is a 6×12 matrix. Section 23.5 separates that logical orientation from the operands' physical address layout. The product has $MN = 72$ output coordinates. Each coordinate sums over $K = 8$ products, so the common multiply-add accounting convention gives $2MNK = 2 \cdot 6 \cdot 12 \cdot 8 = 1152$ scalar floating-point operations. The work count fixes the algebraic effort; the selected kernel and numerical check determine what a timing means. Dtype, memory layout, tensor-core tile shape, JIT state, and routed-row semantics supply those later conditions.

This 6×8 by 12×8 problem is the DeepGEMM scaled stage view. The source-derived architecture lane of the Volume II request trace, REQUEST-TRACE/ARCHITECTURE, supplies realistic V3 projection dimensions. Its scaled systems lane, REQUEST-TRACE/SCALED, supplies a conditional routed row. The observed lane selected SGLang's deep_gemm MoE runner, but its instrumentation stopped at the surrounding routed handoffs. No DeepGEMM kernel launch, JIT specialization, or timing is attributed to the observed request.

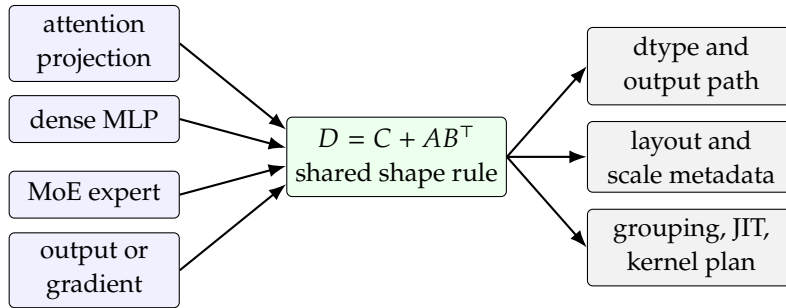


Figure 23.1: Transformer sites share GEMM shape, then diverge in dtype, layout, grouping, JIT state, and kernel plan.

Figure 23.1 maps four transformer call sites to the request fields that distinguish their kernels.

Symbol	Meaning in this chapter
M, N, K	Dense GEMM row count, output-column count, and reduction width.
A, B, C, D	Dense operands and output in $D = C + AB^T$.
E, k_{top}	Number of experts and selected experts per token.
M_e, M_{total}	Valid rows for expert e and total routed rows $\sum_e M_e$.
A_e, B_e, D_e	Expert- e activation block, weight block, and output block.
G, g, K_g	Number of K-axis groups, a group index, and group-specific reduction extent.
A_g, B_g, C_g, D_g	K-grouped slices, accumulator, and output in $D_g = C_g + A_g^T B_g$.
$q, S_A, S_B, K_{\text{block}}$	Quantized payload, operand scale tensors, and scale-block width.
$\tau_{\text{payload}}, \tau_{\text{out}}$	Payload dtype and output dtype.

Table 23.1: Local notation for DeepGEMM dense, low-precision, and grouped GEMM examples.

The reusable symbols appear in Table 23.1. The same letters keep their mathematical meaning when a later section changes the storage dtype, scale layout, or grouping interface.

Definition 23.1.2 (Transformer GEMM site) *A transformer GEMM site is a layer operation that can be written as a batch of activation rows multiplied by a learned projection matrix, or as the corresponding backward product during training. The definition covers attention projections, MLP projections, expert projections, output projections, and training gradients. It does not assert that every site has the same dtype, layout, or batching behavior.*

Table 23.2 compares the row axis and system issue at each site.

Table 23.2.: Transformer GEMM sites share matrix multiplication, but not the same operational constraints.

Site	Example row axis M	Typical N, K meaning	System issue
Attention projection	tokens in a batch	heads and head widths	cache and backend layout
Dense MLP	token rows	intermediate and hidden widths	large dense throughput
MoE expert	routed token-expert rows	expert intermediate width	irregular row counts
Output projection	generated token rows	vocabulary and hidden width	large N dimension
Weight gradient	output/input features	accumulated token rows	training accumulation

For a dense MLP, the six rows of the running example are simply six token activations. For an MoE layer, those same six tokens can become more rows after routing, because one token may be sent to more than one expert. The meaning of M therefore determines whether the implementation reads ordinary token rows, routed token-expert rows, padded rows, or valid rows hidden behind a mask. Section 23.8 makes that row packing and its output check explicit.

Kernel-family orientation. Table 23.3 orders the released surfaces by their role in this chapter. The bands are a learning hierarchy, not a performance ranking: an adjacent kernel can dominate a particular model path while still owing a different output relation from GEMM.

Optimization map. DeepGEMM is not fast because it changes the matrix product. It specializes how represented values move and overlap for one declared product. The six load-bearing ideas appear together in Table 23.4 before later sections open their machinery. The entries are architecture- and shape-dependent choices, not a promise that every kernel uses every mechanism [81].

Chapter 19 supplies the shared kernel vocabulary. DeepGEMM follows a four-verb execution model—tile, stage, accumulate, store—and maps CuTE expressions to six roles: view, layout, tile, fragment, copy, and GEMM. The CUDA sequence in that chapter connects those roles to the execution hierarchy, asynchronous-group controls, and compilation ladder. Only the chapter-specific execution fields needed for the running request remain in Table 23.5.

Band	Kernel families	Primary job and obligation
Foundation	Dense BF16 and FP8/FP4 GEMM	Establish the reference product, represented operands, and architecture path. Every grouped path inherits these shape, dtype, scale, and oracle obligations.
MoE workhorses	<i>M</i> -grouped contiguous/masked and <i>K</i> -grouped contiguous GEMM	Batch variable routed rows for forward work or variable reduction slices for backward-style work. Distinguish which axis varies and which output coordinates remain separate.
Fused handoff	Mega MoE	Reuse the grouped and precision foundations while widening the execution boundary to dispatch, two expert linears, SwiGLU, route weighting, and combine. The persistent SM100 path is taught in Chapter 27.
Adjacent specialized	MQA/indexer logits, finite einsum contractions, and HC prenorm	Produce score, axis-reduced, or paired-reduction outputs. Carry forward JIT and representation ideas, but use each kernel's own oracle.

Table 23.3.: Reader orientation for the principal DeepGEMM kernel families. Foundation and MoE kernels form the main GEMM path; fused and adjacent families extend the optimization boundary or change the output contract.

Pressure	DeepGEMM response	Reader checkpoint
Low-precision metadata	Form a payload partial for one scale block, then promote it with the matching scales into the final accumulator.	Scaling is part of the GEMM pipeline, not a materialized dequantized matrix.
Movement latency	Give TMA producers and matrix consumers different roles and rotate several shared-memory stages.	A later tile can move while an earlier tile computes.
Reuse and tail waste	Keep CTAs persistent, swizzle their logical tile order, admit non-power-of-two block widths, and multicast a reusable operand when a legal cluster exists.	The best tile is not necessarily the largest or a power of two.
General-kernel overhead	JIT-compile the selected shape, layout, architecture, and plan, then cache its cubin.	Cold compilation and warm execution are different events.
Finite registers and shared memory	Give producer and math roles different register budgets and balance tile size, stage count, and cluster reuse.	More stages or a larger tile can reduce rather than improve throughput.
Irregular expert work	Batch valid expert rows with contiguous or masked grouped layouts; hand the multi-rank fused interval to Chapter 27.	Padding and empty experts remain explicit here; dispatch and combine become part of the downstream systems contract.

Table 23.4.: Concept-first map of the DeepGEMM optimization stack. Each response preserves the declared output relation while changing movement, scheduling, or specialization.

Table 23.5.: Execution boundary for the DeepGEMM running request.

Field	DeepGEMM instance
Declared result	$D = C + AB^\top$, with its output shape, accumulation rule, reference, and numerical tolerance.
Execution owner	A GPU work unit owns an output tile, stages the required operand tiles, and updates that tile’s accumulators.
Representation and selection	Payload and scale metadata determine the represented operands; shape, layout, architecture, and JIT state select the compiled kernel plan.
Completion and check	Every K -dimension contribution reaches the accumulators before the converted values are stored to D and compared with the declared reference.

23.2. Baseline GEMM Math and BF16 Reference

Low-precision comparison needs a stable reference calculation. DeepGEMM exports BF16 dense and grouped GEMM functions, and its tests compare kernel outputs with generated reference outputs under source-defined thresholds [81]. The broader numerical-reference and tolerance methodology is developed in Chapter 32; this section fixes only the DeepGEMM operand and output contract.

Definition 23.2.1 (Accumulator and output dtype) *The accumulator is the numeric path used to sum products over the reduction dimension K . The output dtype τ_{out} is the storage dtype of D . An optional input C is a third object: it changes the relation to $D = C + AB^\top$ but is not the internal arithmetic accumulator.*

For the running NT shape, $A \in \mathbb{R}^{6 \times 8}$, $B \in \mathbb{R}^{12 \times 8}$, and $C, D \in \mathbb{R}^{6 \times 12}$. The DeepGEMM BF16 API requires BF16 inputs, permits BF16 or FP32 output, and requires a supplied C to match the output dtype and compatible layout [81]. Those checks define the call boundary; they do not assert an exact hardware addition order or exact real-valued arithmetic.

Example 23.2.1 (One hand-computable dense reference and kernel check) Use a miniature NT instance whose integer entries are represented exactly in BF16:

$$A = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}.$$

Then

$$AB^\top = \begin{bmatrix} 5 & 10 \\ -1 & -4 \end{bmatrix}, \quad D^{\text{ref}} = C + AB^\top = \begin{bmatrix} 6 & 10 \\ -1 & -2 \end{bmatrix}.$$

A DeepGEMM run produces D^{ker} , which is compared with this reference under a declared metric. The retained record names the 2×2 shape, BF16 inputs, optional C , output dtype, concrete inputs, source revision, and comparison threshold. The phrase “the GEMM is correct” is incomplete without those fields.

FP8 and FP4 paths are not judged by bitwise equality against this output. They use the same logical product with source-defined payloads, scales, layouts, and tolerances introduced next.

Input dtype, internal accumulation, optional C , and output dtype are separate parts of the reference GEMM.

23.3. Low-Precision GEMM: FP8, FP4, NVFP4, and Scale Metadata

Low-precision GEMM changes what is stored and moved before it changes the matrix equation. The logical product is still $D = C + AB^\top$. What changes is that A or B may be represented by a compact payload with dtype τ_{payload} and separate metadata needed to interpret that payload. FP8 formats were introduced for deep learning with explicit attention to scaling, range, and accuracy tradeoffs [55]. DeepGEMM's checked source exposes FP8 and FP8/FP4 GEMM functions, but bit-level generic FP4 and NVIDIA FP4 (NVFP4) claims remain hardware-format claims that require the NVIDIA documentation source. The recurring scale-format name is UE8M0: U means unsigned, $E8$ means eight exponent bits, and $M0$ means zero mantissa bits [81, 66].

The general payload-plus-scale definition is developed in Chapter 32. Locally, a represented operand obeys $\hat{x}_{i,j} = s_{i,b(j)} q_{i,j}$: the payload, scale-block map, scale values, and kernel recipe must agree.

Definition 23.3.1 (UE8M0 scale encoding) *UE8M0 is an unsigned eight-bit, exponent-only encoding for scale factors. If $c \in \{0, \dots, 254\}$ is the stored byte, its decoded scale is*

$$s(c) = 2^{c-127}.$$

Thus its finite values are positive powers of two from 2^{-127} through 2^{127} : it has no zero or infinity, and `0xff` is reserved for NaN [66, 59].

UE8M0 spends its byte on exponent range rather than sign or fractional precision. For example, $2 = 2^1$ is stored as $1 + 127 = 128 = \text{0x80}$; 1.5 is not itself representable. DeepGEMM's important obligation is how these codes are packed and assigned to operand blocks, not a second generic scale-selection lesson.

PTX calls a pair of UE8M0 codes `ue8m0x2` and carries the pair in a 16-bit container. That packing rule answers which codes share a container; the tensor layout answers where those containers reside; and scale-block ownership answers which payload entries use each decoded scale. None of the three changes $s(c)$. The Chapter 32 returns to how scale selection and a coarser representable grid affect numerical error.

For the dense running input $A \in \mathbb{R}^{6 \times 8}$, take a toy block width $K_{\text{block}} = 4$. The columns are split into two blocks: columns $1, \dots, 4$ and columns $5, \dots, 8$. A per-row, per-block scale model then has a scale tensor $S_A \in \mathbb{R}^{6 \times 2}$. The first row of the represented tensor is

$$\hat{A}_{1,j} = \begin{cases} S_{A,1,1} q_{1,j}, & 1 \leq j \leq 4, \\ S_{A,1,2} q_{1,j}, & 5 \leq j \leq 8. \end{cases}$$

This example is small enough to inspect. It is not a statement that every DeepGEMM path uses block width 4. DeepGEMM's tests and generators use source-defined recipes such as granularity values of 32 or 128 in the FP8/FP4 paths, and the implementation transforms scale tensors before dispatch [81].

Payload values represent numbers only after the matching scale block is named.

At the recorded source revision, the default SM90 recipe $(1, 128, 128)$ makes that granularity concrete. The first operand has one scale per row and per 128-coordinate K -block; the second has one scale per 128×128 block over its row and K axes. A 1D1D recipe can instead use rowwise scales for both operands, still with 128-coordinate K -blocks. SM100 additionally admits selected 32-coordinate scale recipes. These tuples name scale partitions, not output-tile sizes, and a benchmark must record the chosen recipe rather than treating all FP8 calls as one format [81].

The same toy calculation applies to B . Since B has shape 12×8 in the NT convention, the same block width would give a logical scale tensor $S_B \in \mathbb{R}^{12 \times 2}$. A low-precision GEMM call therefore has more objects than the ordinary formula suggests: payload for A , payload for B , scale metadata for A , scale metadata for B , output D , optional accumulation C , and recipe or layout information. The rule is that payload and scale metadata must jointly represent the values used by the product. Figure 23.2 shows that low-precision GEMM is defined by payload plus metadata rather than by a dtype label alone.

Figure 23.2.: A low-precision GEMM call must carry payloads, scale metadata, and a layout recipe that jointly define the represented values used by the product.

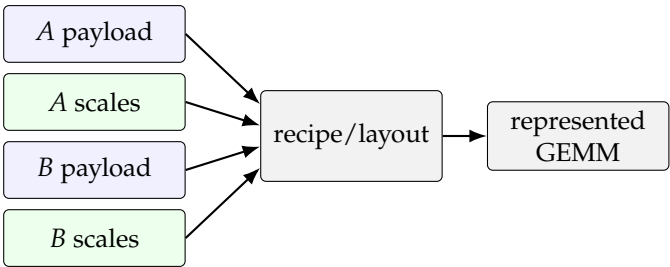


Table 23.6.: Precision paths as represented-value specifications rather than storage names alone.

Path	Textbook object	Required caution
BF16	payload is the stored approximate value	Still not exact real arithmetic
FP8	payload plus scale metadata	Cite FP8 range/scaling claims
FP4/NVFP4	smaller payload plus scale metadata	Cite hardware-format and kernel-source claims
FP8xFP4	mixed operand precision	State which operand and scale recipe are used

Table 23.6 makes the representation obligation explicit. A dtype name is incomplete evidence. An FP8 or FP4 record must also state the scale metadata and reference comparison that make its represented values meaningful. The NVFP4 row is deliberately source-bounded: in this chapter, NVFP4 names a hardware-format FP4 payload path whose represented values still require matching scale metadata, while bit-level layout and alternate-format behavior remain delegated to the NVIDIA PTX source [66].

This representation view also explains why conversion overhead can matter. If a prior kernel already produces FP8 payloads and compatible scales, the GEMM call can consume those objects directly. If an application first stores BF16 tensors and then casts them into FP8 payloads immediately before every call, the conversion becomes part of the measured path. A low-precision speed claim must therefore name the whole path being measured, rather than only the dtype printed in a function name.

23.4. Scaling Factors, Layout, and Accumulation

Scale metadata is mathematical, but a kernel consumes memory. DeepGEMM’s layout API makes this separation explicit: left-hand-side scaling factors must use a Tensor Memory Accelerator (TMA)-aligned and transposed layout. The scaling-factor format also differs between SM90 and SM100: SM90 uses FP32 scaling factors, while SM100 uses the packed UE8M0 scale factors defined in Definition 23.3.1 [81, 66]. UE8M0 determines the decoded numerical scale; packing and the layout transform determine where its code resides in memory. The layout API then performs checks and transformations that convert a logical scale tensor into a compute-required layout.

Here *TMA-aligned* names a physical layout obligation for an eligible TMA path. It does not by itself prove that generated code contains a TMA transfer or that a runtime selected and executed that transfer; those claims require the generated artifact and launch record.

Definition 23.4.1 (Layout) *A layout maps logical tensor indices to memory locations and strides. It is separate from shape and represented value. Two tensors can have the same logical entries while only one satisfies the operand, scale, grouped-work, or internal tile layout required by a selected kernel.*

This general coordinate-to-address map now narrows to scale layout: the payload coordinate and its logical scale must remain paired even when scale entries are transposed, aligned, padded, or packed for an architecture-specific kernel.

For the toy scale tensor $S_A \in \mathbb{R}^{6 \times 2}$, the logical entry $S_{A,3,2}$ means the scale used for row 3 and the second block of columns. A kernel, however, may need those logical entries stored in a different order, aligned to a transaction size, or packed into another format. The mathematical rule is preservation of represented values: after a layout transform, the product must use the same scale assigned to the same logical payload block. Padding and packing are legal only when they do not add real rows or silently change the represented tensor. Figure 23.3 makes that rule concrete: the logical scale table is not the same object as the kernel-facing layout, but it must preserve the scale assigned to each payload block.

A scale tensor can have the right logical values and still be illegal if the kernel-required memory layout differs.

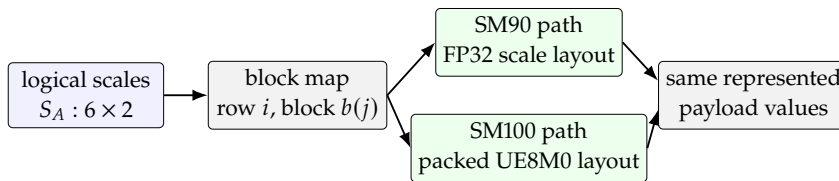


Figure 23.3: A scale-layout transform changes the memory object consumed by the kernel, while preserving which scale belongs to each logical payload block.

Definition 23.4.2 (Packed UE8M0 scale word) *On the SM100 layout path, DeepGEMM stores four UE8M0 exponent codes in the four byte lanes of one 32-bit word. DeepGEMM’s packing transform accepts FP32 powers-of-two whose sign and mantissa bits are zero, extracts each exponent byte, and packs the four bytes without changing which payload block owns each scale [81]. Packing is therefore a storage transform, not a second quantization rule.*

For example, the four scales $[\frac{1}{2}, 1, 2, 4]$ have exponent codes $[7e, 7f, 80, 81]$. The recorded bit shifts write them as

`0x81807f7e,`

with 7e in the least-significant byte lane. Unpacking those lanes recovers the same four powers of two. Reversing their byte order would be a layout error even though all four individual codes remain legal.

The optional C addend changes the output relation to $D = C + AB^T$; it is not the internal accumulator. DeepGEMM requires C to match the output shape and dtype, while the selected kernel separately owns the product’s hardware accumulation path [81]. The general accumulator/output distinction is defined in Definition 23.2.1.

DeepGEMM also exposes a partial-sum layout. Grouped GEMM calls select it with `use_psum_layout`. Their tests record partial-sum offsets, verify zero padding, and transform scale layouts for contiguous and masked groups [81]. This mode differs from the optional C addend. Three layers remain separate: the mathematical output $D = C + AB^T$, the internal product accumulator, and the storage used to schedule irregular expert work.

Table 23.7 keeps the mathematical objects beside the layout and dtype obligations that make them kernel-legal.

Table 23.7.: Logical GEMM objects and kernel-facing obligations.

Object	Logical textbook view	Kernel-facing obligation
A payload	$M \times K$ operand	Dtype and major layout must be legal
B payload	$N \times K$ operand	Transposed interpretation must match call name
S_A, S_B	scale tensors for payload blocks	Transform or pack into required scale layout
C	optional $M \times N$ addend	Same output dtype and compatible layout
D	$M \times N$ output	BF16 or FP32 where the API permits it

A shape error, a scale-layout error, and an output-dtype error may all appear near the same GEMM call, but each points to a different caller obligation.

Table 23.8.: Failure checks that separate valid algebra from a valid DeepGEMM call.

Load-bearing fact	Example invalid call	Later pressure point
Scale metadata matches payload blocks	Payloads have valid 6×8 and 12×8 shapes, but S_A has one column for a two-block recipe	Low-precision correctness cannot be interpreted
Scale layout matches the architecture path	Logical scales are correct, but the caller skips the required transformed or packed layout	The kernel preconditions fail before timing has meaning
Output and C dtypes agree with the API	The algebraic C has shape 6×12 , but its dtype does not match the requested output path	BF16 or FP8 comparison records name the wrong object

Table 23.8 supplies concrete invalid-call cases under the source-backed API discipline [81]. Each row can have the ordinary $D = C + AB^T$ shapes and still fail because the represented kernel object is incomplete or inconsistent.

This is the point at which a mathematically valid product can become an invalid call. A user may have $A \in \mathbb{R}^{6 \times 8}$ and $B \in \mathbb{R}^{12 \times 8}$, but an FP8/FP4 DeepGEMM call still needs payload dtypes, scale tensors, scale recipes, and layouts that match the chosen architecture path. For grouped paths, the same principle applies to grouped payloads and grouped scale metadata. The useful mental model is this: the kernel implements GEMM over represented values, subject to explicit dtype, scale, layout, and accumulation rules.

23.5. DeepGEMM Call Interface

With the GEMM equation, BF16 reference, low-precision representation, and scale layout fixed, the package surface becomes an implementation map. DeepGEMM exposes BF16, FP8, FP4, grouped GEMM, MQA scoring, and Mega MoE operations through a Python package backed by C++/CUDA bindings [81]. Their shared boundary binds logical operands and scale metadata to shape, dtype, layout, architecture, and output checks before a JIT kernel can run.

The call interface realizes the shape, representation, scale-ownership, and accumulation contracts already established.

This section inventories the Mega MoE binding only as a DeepGEMM package surface. Its symmetric-memory ownership, current task scheduler, fused epilogues, completion protocol, and evidence boundary belong to Chapter 27.

The call path has three levels. Python selects a named operation. C++ bindings check shape, dtype, layout, and optional arguments. The JIT then selects a CUDA kernel for the GPU architecture and scale metadata. Figure 23.4 shows that path.

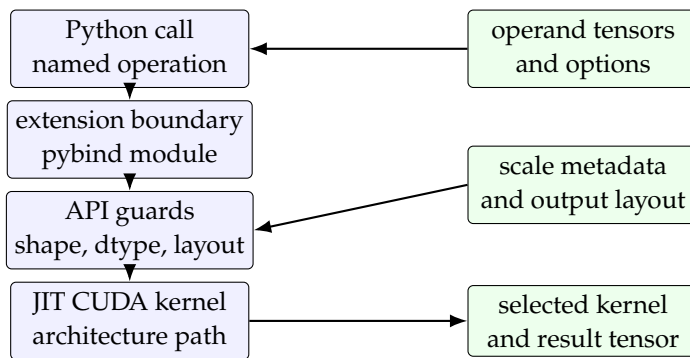


Figure 23.4. DeepGEMM call path from a Python operation through checked arguments to a JIT-selected CUDA kernel.

Concept-to-code map

The extension boundary begins in <https://github.com/deepseek-ai/DeepGEMM/blob/1f6f3f378920ccb5cc036ef43eb3f5972e921713/setup.py> and https://github.com/deepseek-ai/DeepGEMM/blob/1f6f3f378920ccb5cc036ef43eb3f5972e921713/csrc/python_api.cpp; the dense call contract continues in

<https://github.com/deepseek-ai/DeepGEMM/blob/1f6f3f378920ccb5cc036ef43eb3f5972e921713/csrc/apis/gemm.hpp> [81].

- **Extension boundary.** The build declares a CUDAExtension named `deep_gemm._C`; the pybind initializer registers the exported function groups.
- **Dense and low-precision GEMM.** The call surface includes `bf16_gemm_nt`, `fp8_fp4_gemm_nt`, and grouped GEMM entry points. The attention API also binds the head-split store transform through `fp8_gemm_nt_skip_head_mid` and `EpilogueHeadSplits`.
- **Scale layout.** Layout helpers transform FP8 and FP4 scaling factors into the form consumed by the selected kernel.
- **JIT and fused expert paths.** Runtime compilation, PTX wrappers, Mega MoE bindings, and generated-shape helpers choose and validate specialized kernels.
- **Numerical checks.** BF16, FP8, FP4, generated-shape, and Mega MoE tests use `calc_diff` and reference computations.

At the source-inspection snapshot recorded in Appendix A, package version 2.6.1 imports the pybind module `deep_gemm._C`; no dispatcher registration is implied. The fixed H200 measurement rows later in the chapter remain attached to the earlier measured commit 54e226124093. They do not retroactively validate the tensor-map race repair added by the current inspection pin. That repair remains an inspection-level source-history boundary here; it is not presented as one of the chapter’s worked kernel algorithms.

Definition 23.5.1 (Kernel call contract) *A kernel call contract is the set of logical shapes, dtypes, layouts, optional arguments, architecture assumptions, and output conventions that must hold for a kernel call to be meaningful. It is stricter than the mathematical GEMM definition because a mathematically valid product may still violate a kernel’s layout or dtype requirements.*

For the running dense example, Definition 23.1.1 says that a 6×8 matrix can multiply a 12×8 matrix transposed to produce a 6×12 output. DeepGEMM adds the convention $D = C + A @ B$ with $[M, K] @ [N, K]^T$, plus dtype and layout guards [81]. The BF16 path accepts BF16 operands and BF16 or FP32 outputs. FP8 and FP4 operands travel with scale tensors, whose factors are rearranged before dispatch.

Shape, layout, and orientation. A shape says which logical coordinates exist. A kernel must additionally know which address holds each coordinate. That distinction is load-bearing here: two tensors can print as the same matrix while presenting different strides and therefore different legal access paths to a kernel.

Definition 23.4.1 names that coordinate-to-address map before the scale-layout obligations that first use it.

For a concrete address example, let

$$X = \begin{bmatrix} x_{0,0} & x_{0,1} & x_{0,2} \\ x_{1,0} & x_{1,1} & x_{1,2} \end{bmatrix}.$$

Table 23.9 traverses the same six logical entries in two address orders. Offsets begin at zero and count elements rather than bytes.

Storage order	Consecutive offsets $0, \dots, 5$	Strides for (i, j)
Row major	$x_{0,0}, x_{0,1}, x_{0,2}, x_{1,0}, x_{1,1}, x_{1,2}$	$(3, 1)$
Column major	$x_{0,0}, x_{1,0}, x_{0,1}, x_{1,1}, x_{0,2}, x_{1,2}$	$(1, 2)$

Table 23.9.: One logical 2×3 tensor under row-major and column-major address orders. Shape and values stay fixed while the coordinate-to-offset map changes.

DeepGEMM’s two-letter suffix applies one letter to each input. N means that input enters the written product without a logical transpose; T means that it enters transposed. Table 23.10 makes the passed-shape convention concrete.

Suffix	Passed operand shapes	Written product
NT	$[M, K], [N, K]$	AB^T
NN	$[M, K], [K, N]$	AB
TN	$[K, M], [K, N]$	$A^T B$
TT	$[K, M], [N, K]$	$A^T B^T$

Table 23.10.: DeepGEMM dense orientation suffixes. Each letter names how the corresponding passed operand enters the written product; it does not assert that a material transpose is free.

A transpose view can change shape and strides without moving bytes; a material transpose writes a new memory order. The suffix therefore describes the call’s orientation contract, not a promise that preparing it is free [81].

The word *layout* has four scopes in the rest of the chapter:

Operand layout

The major order and strides through payloads A and B , including the N/T orientation contract.

Scale layout

The packing, alignment, and address order of low-precision metadata S_A, S_B .

Grouped-work layout

The segment, mask, padding, or partial-sum records that identify real expert rows.

Internal tile layout

The shared-memory swizzle, fragment distribution, or tensor-memory columns used inside one architecture-specific kernel plan.

Every use of “layout” below identifies which of these four objects is being arranged.

SM90 and SM100. Here SM90 and SM100 name GPU architecture targets, not the number of streaming multiprocessors in a device. The SM90 path is the Hopper-oriented target used by the chapter’s H200 measurements; SM100 is the Blackwell SM100 target. These labels do not cover every GPU in a marketing generation. The two paths preserve the same GEMM equation but admit different operand orientations, scale representations, staging plans, and matrix-operation machinery [81, 66].

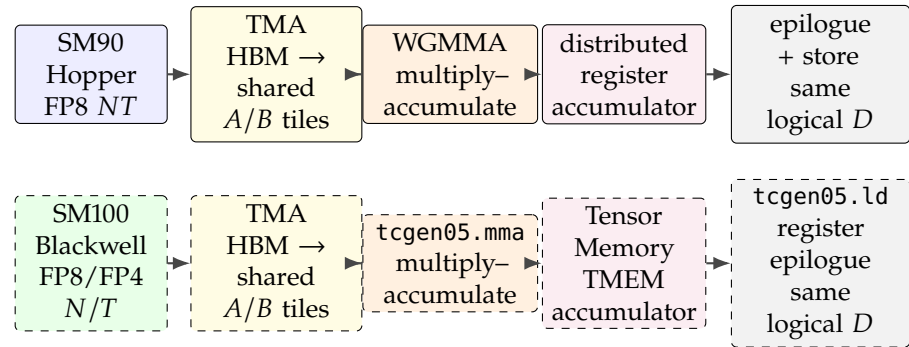
The names become easier when each receives one verb. The Tensor Memory Accelerator (TMA) *moves* multidimensional tiles between global and shared memory. The transaction-backed *mbarrier* definition and six-step trace in *A Reusable TMA/WGMMA Transaction Pipeline* in Chapter 19 explain how a staged TMA transfer becomes readable and why phase parity matters when its shared buffer is reused. That byte-readiness fact does not complete the later matrix operation. *WGMMA, TMA, Swizzles, and Tensor Memory* defines WGMMA’s $D \leftarrow AB + D$

update, distributed accumulator, and the SM100 Tensor Memory allocation–compute–extract–deallocation lifecycle; *Completion, Publication, Eligibility, and CUDA Graphs* separates the fence–issue–commit–wait states.

Tensor Memory (TMEM) *holds* Blackwell matrix state; despite the shared words, it is not TMA and is not ordinary shared memory. On SM90, warpgroup matrix multiply-accumulate (WGMMMA) operations *compute* into distributed register accumulators. On SM100, the fifth-generation Tensor Core `tcgen05.mma` family computes $D \leftarrow AB + D$ into TMEM. After the matching completion, a `tcgen05.ld` extracts the completed tile into registers for the epilogue and final store. DeepGEMM’s CuTe-facing source also uses UMMA names for Blackwell layout and operation objects; the inline PTX wrapper supplies the concrete `tcgen05.mma` spelling.

This is a deliberate boundary, not a rejection of CUDA C++ or CUTLASS/CuTe. DeepGEMM keeps kernel structure in CUDA C++ and uses CUTLASS/CuTe for layouts, descriptors, barriers, and supported matrix atoms. It drops to inline PTX when the recorded implementation needs an exact architecture instruction, memory- ordering scope, asynchronous-group control, or operand form that those higher layers do not conveniently expose. The embedded PTX remains source intent: it still passes through the compiler, and only cubin-derived SASS records the machine code ultimately emitted. It is therefore distinct from the optional whole-kernel PTX dump discussed in *Runtime JIT, Plan Selection, and Cold/Warm Measurement*. These mechanisms change the storage and instruction path, not which logical D the call must return [81, 66, 67].

Figure 23.5.: Conceptual DeepGEMM architecture split. The legal representations and accumulator homes differ: SM90 WGMMMA updates distributed register fragments, while SM100 TCGen05 updates TMEM and later extracts the result for the epilogue. Both paths owe the same logical output and reference check. The diagram is a contract map, not a cycle-accurate pipeline.



The common operand-movement stage ends in shared memory; the branch begins at the matrix operation and accumulator home. The rightmost boxes reunite at the same logical D , so neither architecture name changes the API-level oracle.

Example 23.5.1 (One SM100 accumulator across two K -slices) Figure 23.5 supplies the SM100 storage path instantiated by this trace. Reuse the 4×4 teaching GEMM and CTA (1,0) from Systems Foundations, now on the conceptual SM100 branch. This CTA still owns rows 2,3 and columns 0,1 of $D = C + AB^\top$. After allocating TMEM for its accumulator and initializing that logical tile from C , the first staged reduction slice produces

$$D^{(1)} = C_{2:4, 0:2} + A_{2:4, 0:2} B_{0:2, 0:2}^\top$$

in TMEM. The second slice updates the same storage:

$$D^{(2)} = D^{(1)} + A_{2:4, 2:4} B_{0:2, 2:4}^\top.$$

Only after the matching TCGen05 completion may `tcgen05.ld` extract $D^{(2)}$ into registers for the epilogue and store. The TMEM region remains live until that extraction and its affected users finish, after which it can be deallocated. The 2×2 trace is not a legal TCGen05 shape or TMEM allocation. Real SM100 kernels use supported physical layouts but preserve the same rule: every K -slice updates one logical accumulator before extraction [66, 67].

The checked package requires NVIDIA SM90 or SM100 GPUs, Python 3.8 or newer, C++20-capable compilers, a compatible CUDA Toolkit, PyTorch 2.1 or newer, CUTLASS 4.0 or newer, and `fmt` [81]. These conditions are part of kernel eligibility: a valid matrix product must use a fallback when the installed system cannot satisfy them. FP8 format terminology follows the standard FP8 treatment [55], with architecture-specific alternate-format details supplied by NVIDIA PTX documentation [66].

Exported surfaces and architecture eligibility The Python module exports more names than one device can necessarily execute. The four dense suffixes use the N/T convention defined above; they are not permission to pass an arbitrary strided tensor. The scaled low-precision SM90 path implements the NT memory layout, while the SM100 path implements NT, TN, NN, and TT. BF16 has a broader wrapper surface, shown separately below. The corresponding wrappers form transpose views where appropriate, but the caller still owns any material transpose, cast, or fusion required to produce an eligible memory layout [81].

Table 23.11.: Architecture eligibility for the core dense and grouped GEMM surfaces. A Python symbol does not erase its layout, dtype, metadata, or device preconditions.

Exported family	SM90 boundary	SM100 boundary	Caller-owned state
Dense BF16 GEMM	NT, NN, TN, and TT wrappers dispatch through the SM90 path after shape and stride checks	The same four wrapper orientations dispatch through the SM100 path	Logical orientation, legal major strides, optional C, and output buffer.
Dense FP8/FP4 GEMM	FP8 NT path with both inputs K-major; FP32 low-precision scales; no FP4 path	FP8/FP4 NT, TN, NN, and TT paths with packed UE8M0 scales	Logical orientation, material layout, casting, scale construction, optional C, and output buffer.
M-grouped GEMM	Contiguous and masked NT families for supported BF16/FP8 recipes	Contiguous NT/NN and masked NT families for supported BF16/FP8/FP4 recipes	Expert-row packing, valid counts or mask, alignment, padding, scales, and grouped map.
K-grouped GEMM	BF16 TN and FP8 NT contiguous families; explicit aligned K_g lengths and no partial-sum layout	BF16 and FP8 TN contiguous families; aligned lengths or partial-sum end offsets	Concatenated reduction slices, group extents, FP32 C/D , scale metadata, padding, and output group order.

Table 23.12.: Architecture eligibility for adjacent and fused DeepGEMM surfaces. These entry points have their own output semantics and metadata contracts in addition to architecture gates.

Exported family	SM90 boundary	SM100 boundary	Caller-owned state
Einsum	Hard-coded BF16 contractions and the SM90-supported FP8 contraction set	Same BF16 set plus the SM100-only FP8 transpose/gradient contractions	Exact expression string, batch/head axes, accumulator/output dtype, and scales.
Head-split FP8 GEMM	FP8 NT under the SM90 scale-recipe guard	FP8 NT with packed UE8M0 scales	The (L, H, R) split, widened output, legal store-block divisibility, and ownership of unwritten middle columns.
MQA/indexer logits	FP8 query/KV paths; SM90-specific head/page restrictions	FP8 or packed FP4 query/KV paths, more head counts, page sizes, and optional varlen indices	Query/KV quantization, weights, logical lengths, page table, schedule metadata, and output cleaning.
HC prenorm	TF32/BF16-input implementation	TF32/BF16-input implementation	FP32 prenorm matrix, optional split outputs, and explicit reduction across splits.
Mega MoE	Unsupported by the pinned fused implementation	Released fused SM100 path	Symmetric buffers, route state, transformed weights, and multi-process launch.

Definition 23.5.2 (Head-split GEMM epilogue) *The exported `fp8_gemm_nt_sk_ip_head_mid` computes an FP8 NT product with logical width N , but scatters its columns into a wider output that reserves a middle span. For split widths (L, H, R) , it requires N to be divisible by $L + R$, gives the output width*

$$N' = N + \frac{N}{L + R}H,$$

and maps source column n to

$$n' = n + \left\lfloor \frac{n + R}{L + R} \right\rfloor H.$$

The specialized CUDA epilogue performs this index transform during the GEMM store. It does not compute or initialize the reserved middle columns [81].

For example, in store-block units, take $(L, H, R) = (2, 1, 2)$ and eight computed columns. The mapping is

$$(0, 1, 2, 3, 4, 5, 6, 7) \mapsto (0, 1, 3, 4, 5, 6, 8, 9),$$

so output columns 2 and 7 remain reserved. The actual template additionally requires L, H , and R to be divisible by the selected store-block width; the unit-scale trace explains the indexing rather than proposing a legal scalar launch. Folding the scatter into the epilogue removes a separate rearrangement pass, while making the widened-output and gap-ownership contract part of correctness.

Tables 23.11 and 23.12 are eligibility maps rather than performance rankings. They also separate DeepGEMM work from adjacent preprocessing: the repository explicitly leaves input transposition and low-precision casting to the caller or a prior fused kernel. A benchmark that times only the GEMM kernel must not silently claim that those transformations were free.

TileKernels is adjacent but implements a different layer. Its TileLang kernels cover MoE gating and routing, quantization casts, transpose, Engram, and Manifold HyperConnection paths [45]. Those kernels may prepare rows or metadata around a DeepGEMM call, but they do not replace the DeepGEMM shape, dtype, scale-layout, or JIT contract.

23.6. Runtime JIT, Plan Selection, and Cold/Warm Measurement

Cold and warm JIT paths. DeepGEMM compiles its kernels at runtime through a lightweight JIT module, and installation does not require CUDA compilation [81]. That makes the first call and a warm repeated call feel different for a reason. A first call can include compilation, cache creation, and kernel loading. A later call for the same effective signature may reuse a cached runtime object. A benchmark that mixes those cases without saying so is not measuring one well-defined event.

Table 23.13.: A smaller, non-power-of-two width can improve wave fill for an awkward shape. This arithmetic compares available tile work, not measured latency.

Block N	Output-tile grid	First-wave useful CTAs	Wave fill with 132 SMs
128	2×56	112	$112/132 \approx 84.8\%$
112	2×64	128	$128/132 \approx 97.0\%$

Definition 23.6.1 (JIT specialization) *JIT specialization is runtime compilation or loading of a kernel selected for a particular combination of operation, shape, dtype, layout, architecture, compiler settings, and selected compile dimensions. The resulting kernel is a runtime artifact, more than a Python function name.*

Why JIT belongs to the fast path. The JIT specialization in Definition 23.6.1 moves selected choices from runtime state into the generated artifact. A general kernel must preserve choices until launch: dimensions may remain runtime values, loops may need bounds, and one body may carry branches for several legal plans. DeepGEMM instead lets selected dimensions and plan fields become constants in generated source. The compiler can then unroll the fixed matrix pipeline, remove unreachable cases, and allocate registers for the chosen tile and thread roles. This is especially useful when a small or awkward shape cannot amortize general-kernel bookkeeping. The cost is shifted to the first call; the cubin cache is what makes the specialized choice practical for later calls [81].

The selection step is deterministic rather than a hidden timing search at the recorded revision. Architecture-specific heuristics enumerate legal block, cluster, and stage candidates, reject candidates that violate resource or layout constraints, and compare the survivors with a source-defined cost model. JIT then compiles the selected plan; it does not benchmark every candidate on the user's request path. The useful mental model is therefore a small kernel factory: the call signature chooses a plan, the plan becomes source constants, and the digest-named artifact makes that choice reusable.

The effective signature selects a legal plan; the JIT cache decides whether that already-selected plan must be compiled.

Example 23.6.1 (Why a non-power-of-two block width can improve wave fill) Consider $M = 256$, $N = 7168$, block $M = 128$, and a budget of 132 SMs. With block $N = 128$, the grid contains

$$(256/128)(7168/128) = 2 \cdot 56 = 112$$

output tiles. With the legal candidate block $N = 112$, it contains

$$(256/128)(7168/112) = 2 \cdot 64 = 128$$

tiles. The second plan can place useful first-wave work on 128 rather than 112 SMs. Here *non-power-of-two* describes the logical output-tile width; it does not waive TMA alignment or shared-memory swizzle requirements.

Table 23.13 keeps the two candidate grids and their first-wave utilization beside the example.

The example explains why the heuristic can admit tile widths tied to hardware and user-configured multiples instead of restricting the search to powers of

two. It is not proof that block $N = 112$ wins: the selected plan must also pay for its extra tiles and satisfy register, shared-memory, swizzle, stage, multicast, and data-movement constraints. The source-defined cost model compares those surviving plans and includes wave efficiency [81].

Definition 23.6.2 (Retained SM90 selector cost) *For a legal SM90 candidate $p = (B_M, B_N, C_M, C_N, \text{stages})$, let g be the expected group count, S the SM budget, and*

$$Q(p) = \left\lceil \frac{M}{B_M} \right\rceil \left\lceil \frac{N}{B_N} \right\rceil g, \quad W(p) = \left\lceil \frac{Q(p)}{S} \right\rceil, \quad \eta(p) = \frac{Q(p)}{W(p)S}.$$

Write $s_{A,B}$ and $s_{C,D}$ for input and output element bytes, and set $\alpha = 2$ when an existing accumulator is read and 1 otherwise. The retained source estimates per-block L2 and L1 traffic as

$$b_2 = K \left(\frac{B_M}{C_N} + \frac{B_N}{C_M} \right) s_{A,B} + \alpha B_M B_N s_{C,D},$$

$$b_1 = K(B_M + B_N)s_{A,B} + K(\max(64, B_M) + B_N)s_{A,B} + (1 + \alpha)B_M B_N s_{C,D}.$$

With source constants $\beta_2 = \min(64S, 8 \times 10^6 / 1.3 \times 10^3)$ and $\beta_1 = 128S$ bytes per cycle, the deterministic rank is

$$c_{\text{SM90}}(p) = \frac{\max\{Q(p)b_2/\beta_2, Q(p)b_1/\beta_1\}}{\eta(p)}.$$

Lower is preferred. A multi-CTA cluster is assigned infinite cost when it would occupy no more than one wave. Candidate stages, register use, shared memory, layout, and cluster shape enter through the preceding legality filter rather than as additional terms in this equation [81].

Definition 23.6.2 exposes what “cost model” means for the retained SM90 path. The cluster divisors model multicast reuse; the two traffic estimates compete through a maximum; and $1/\eta$ penalizes an incompletely filled final wave. It is a deterministic ordering rule, not a latency prediction or request-path autotuning result. Architecture-specific selectors may use a different rule.

Implementation lens: descriptor to selected JIT launch

The dense SM90 BF16 host path turns the call into a descriptor, selects one legal plan, renders source, resolves the compiled runtime, and launches it. The comments marked “book elision” replace the descriptor fields and tensor-map binding retained in the full source.

```
1 const auto desc = GemmDesc {
2     .gemm_type = GemmType::Normal,
3     .kernel_type = KernelType::KernelNoSF,
4     .m = m, .n = n, .k = k, .num_groups = 1,
5     .a_dtype = a.scalar_type(), .b_dtype = b.scalar_type(),
6     .cd_dtype = d.scalar_type(),
7     // ... book elision: layouts, accumulation, SM budget ...
8     .tc_util = device_runtime->get_tc_util(),
9     .compiled_dims = compiled_dims
```

```

10 };
11 const auto config = get_best_config<SM90ArchSpec>(desc);
12 // ... book elision: tensor maps and launch Args ...
13 const auto code = SM90BF16GemmRuntime::generate(args);
14 const auto runtime = compiler->build(
15     "sm90_bf16_gemm", code);
16 SM90BF16GemmRuntime::launch(runtime, args);

```

Descriptor semantics. The descriptor is the semantic and hardware eligibility record; the selected `config` is the deterministic plan. Generation freezes that plan into source, `build` resolves a cached or newly compiled cubin, and `launch` executes the loaded runtime. A warm cache hit may skip compilation, but it does not skip descriptor validation or plan identity.

This abridged excerpt comes from DeepGEMM revision 1f6f3f378920, under the repository's MIT License [81]. The public companion records the exact upstream source coordinate in `chapter_deepgemm/README.md`. The excerpt exposes the ownership transitions; it does not show the full cache-publication protocol or establish which plan is fastest on another signature.

For the dense running example, a call such as a BF16 *NT* GEMM is not identified only by $M = 6$, $N = 12$, and $K = 8$. It also depends on the architecture family, operand layouts, output dtype, whether *C* is present, compiler choice, and the `compiled_dims` argument exposed by the API. DeepGEMM's JIT compiler source builds cache paths from a kernel name, compiler signature, flags, and generated code; it also supports a configurable cache directory through `DG_JIT_CACHE_DIR` [81]. Thus the shape variables are only part of the specialization key.

On a cache miss, DeepGEMM first renders CUDA source for the selected kernel plan. By default it asks NVCC to compile that source directly to a cubin; with `DG_JIT_USE_NVRTC`, it instead uses NVRTC, NVIDIA's in-process runtime CUDA compiler. The NVRTC path returns the target cubin and can additionally retain PTX when a dump is requested.

PTX is a virtual GPU instruction format; when it is an intermediate artifact, PTXAS, NVIDIA's PTX assembler and optimizing back end, lowers it to the target architecture's machine code. A cubin is the resulting CUDA device binary that DeepGEMM places in the digest-named cache directory and loads as a kernel runtime. The exposed intermediate files differ, but both compiler paths turn generated source into the architecture-specific binary reused by a warm launch [81, 66].

Cache publication is also a concurrency boundary. DeepGEMM compiles in a unique temporary digest directory, synchronizes the generated files and directory, and then attempts an atomic rename to the shared cache path. If another process publishes the same digest first, the loser discards its temporary directory and loads the winner's complete artifact. Thus no reader should observe a partially written cubin. Chapter 26 visualizes and failure-tests the same publish-then-load JIT-cache pattern.

Figure 23.6 turns that key into the timing boundary used throughout the section. A cache hit and a cache miss may launch the same mathematical GEMM, but they are different measured events.

Algorithm 6: Deterministic DeepGEMM plan selection and JIT-cache resolution.

Input: Effective signature σ ; candidate generator G ; legality predicate L ; deterministic cost c ; JIT cache H

Output: Kernel result or typed compile/load/launch failure, with the selection/cache record

```

1 enumerate candidate plans  $\mathcal{P} \leftarrow G(\sigma)$ ;
2 discard every  $p \in \mathcal{P}$  for which  $L(p, \sigma)$  is false;
3 if  $\mathcal{P} = \emptyset$  then
4   return unsupported signature with rejection evidence;
5 choose  $p^* \leftarrow \arg \min_{p \in \mathcal{P}} c(p, \sigma)$  with the source-defined deterministic tie break;
6 render source constants for  $(\sigma, p^*)$  and form the digest from the kernel name, compiler signature,
  flags, and generated source;
7 if the digest exists in  $H$  then
8   mark the event warm and attempt to load the cached cubin;
9   if cache loading fails then
10    return typed cache-load failure with signature, plan, digest, and compiler;
11 else
12   mark the event cold, compile in a unique temporary directory, synchronize its files, and
    attempt an atomic rename to the digest path;
13   if another process wins publication, discard the temporary directory and attempt to load the
    winner's cubin;
14   if compilation, publication, or cubin loading fails then
15    return typed JIT failure with signature, plan, digest, compiler, and cold class;
16 create the kernel runtime from the resolved cubin;
17 if runtime creation fails then
18   return typed runtime-load failure with the selection/cache record;
19 launch it with  $\sigma$  and retain launch status;
20 if the launch fails then
21   return typed launch failure with the selection/cache record;
22 return the result plus signature, selected plan, digest, compiler, and cold/warm class;

```

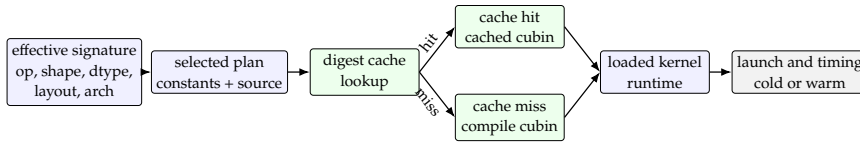


Figure 23.6.: DeepGEMM selects a kernel plan, renders its fixed choices as source constants, and uses the source-dependent digest to look up a cubin. A miss compiles the cubin; both paths load a kernel runtime before launch.

Algorithm 6 first filters candidate plans by architecture, layout, and resource legality before applying the deterministic cost model. Then it renders the selected choices into source and resolves the source-dependent digest through either the warm or cold path. Finally, it creates and launches the loaded runtime. Compile, cache-load, runtime-load, and launch failures terminate with the same selection/cache evidence; only a successful launch returns a kernel result. The safety boundary is that an illegal or unsupported signature never falls through to a convenient kernel, and a cold compilation or failure is never mislabeled as warm kernel latency.

Example 23.6.2 (Separating first-call and steady-state timing) Suppose an experiment measures the running dense BF16 example. A reproducible protocol should record one timing for first-call compile/load behavior and a separate timing for warm steady-state calls. The sampled H200 rows below fill both classes for two fixed signatures. The two numbers answer different questions.

Table 23.14 names those two measurement classes before either row is interpreted as `latency`.

Table 23.14.: Cold-call and warm-call timing answer different benchmark questions.

Event	What may be included	Measurement class
First call	Signature resolution, generated source compilation or load, cache creation, kernel launch, and synchronization	warmup timing
Warm call	Cache hit or reused runtime artifact, kernel launch, execution, and synchronization under the same effective signature	steady-state timing

Cold and warm events answer distinct timing questions. A latency number is interpretable only after the measured event is identified.

Table 23.15.: Minimum JIT-related fields for a timing record.

Field	Why it matters
DeepGEMM commit	Identifies generated source and API behavior
GPU architecture	Determines SM90, SM100, or unsupported dispatch path
CUDA and compiler	Affects JIT legality and generated code
Shape and layout	Determines compiled or selected kernel variant
Warmup policy	Separates compile/load from repeated kernel timing
Cache directory state	Changes whether the call hits or misses JIT cache

Table 23.15 gives the minimum reproducibility record. Omitting the cache state or warmup policy can make a compilation measurement look like a kernel measurement, or the reverse.

The retained H200 closeouts show the practical gap between first-call and warm execution. Their exact fixed-signature timings, cache transitions, sampled steady-state rows, profiled specialization, and counter blocker are consolidated in Chapter 35. Its DeepGEMM dossier keeps the profiled specialization beside the first-call, warm, and sampled H200 rows. The distinction needed here is methodological: first-call host wall time includes generation, compilation or load, and a launch, whereas warm CUDA-event timing reuses the selected specialization. Dividing those instruments does not isolate compiler cost.

23.7. One Generated SM90 Specialization

The retained artifact makes the optimization stack concrete. Its generated instantiation leaves M dynamic, fixes $N = 2112$ and $K = 7168$, and selects a $128 \times 192 \times 128$ CTA tile with four shared-memory stages, 128 TMA threads, 256 math threads, two-CTA multicast on B , and 132 persistent CTA slots. The resulting $32 \times 11 = 352$ output tiles are scheduled over the H200's SMs rather than assigned one resident CTA for the lifetime of the launch. These plan constants belong to the retained 54e226124093 artifact; the same source mechanisms were rechecked at the chapter's later recorded revision [81].

Concept-to-code map

The retained generated CUDA artifact makes the selected constants visible in the template instantiation:

```

1 static void __instantiate_kernel() {
2     auto ptr = reinterpret_cast<void*>(
3         &sm90_fp8_gemm_1d2d_impl<
4             cute::UMMA::Major::K,
5             0, 2112, 7168,
6             1,
7             128, 192, 128,
8             128, 128, 128,
9             4,
10            128, 256,
11            2, false,
12            132, GemmType::Normal,
13            cutlass::bfloat16_t,
14            epilogue::transform::EpilogueIdentity
15        >);
16 };

```

- **Generated plan constants.** The arguments map dynamic M , fixed N , K , CTA tile, stages, thread roles, multicast, and persistent slots to the retained 54e226124093 artifact.
- **Reviewed identity.** The coordinate and SHA-256 map to chapter_deepgemm/README.md; cubin, launch, and numerical records remain separate evidence.

Definition 23.7.1 (Persistent tile schedule and TMA multicast cluster) *Let S be the selected number of persistent CTA slots. Physical slot p claims logical work indices $p, p + S, p + 2S, \dots$ while those indices remain inside the tile set; the schedule swizzle then maps each claimed index to an output tile. Separately, a legal two-CTA cluster may let one elected CTA issue a TMA transfer of a shared operand into both CTAs' shared-memory destinations. A cluster along N shares an A tile; one along M shares a B tile. Multicast shares an operand transfer, not output ownership: each CTA retains its own output tile, nonshared operand, barriers, and completion obligations. A missing or illegal peer falls back to a non-multicast transfer on the recorded SM90 path [81].*

Persistent slot p schedules multiple logical tiles; multicast may share an operand but never output ownership.

Example 23.7.1 (The retained 352-tile schedule) For the retained plan, $S = 132$ and there are 352 logical output tiles. Slot 17 claims indices 17, 149, 281; its next candidate, 413, is outside the tile set. Slot 100 claims only 100, 232. Thus slots 0–87 receive three logical indices and slots 88–131 receive two. For multicast on B , the retained source compares grouping widths $G \in \{8, 16\}$ with the reuse proxy

$$GB_M + \left\lceil \frac{S}{G} \right\rceil B_N.$$

Here $G = 8$ gives 4288, whereas $G = 16$ gives 3776, so the swizzle groups 16 consecutive M -blocks. With 32 M -blocks and 11 N -blocks, one group spans $16 \cdot 11 = 176$ logical indices. For index t , let $h = \lfloor t/176 \rfloor$ and $u = t \bmod 176$; then

$$(m(t), n(t)) = (16h + (u \bmod 16), \lfloor u/16 \rfloor).$$

The selected two-CTA multicast on B pairs CTAs that own different M -tiles over the same N -range. They can receive one shared B tile, but they still load different A tiles and write different regions of D . The persistent stride decides which work comes next; the cluster decides whether one operand movement can be shared. Neither decision changes the GEMM output relation.

Table 23.16 resolves the claimed indices into output coordinates. The final two rows isolate the SM90 odd-tail rule: when a two-CTA multicast group would contain five primary tiles, the swizzle uses an even four-tile prefix and schedules the fifth tile as unicast. The tail therefore preserves output coverage without inventing a multicast peer.

Table 23.17 separates the benefit of each plan choice from the invariant and resource check that still governs it.

The next example moves inside one of those claimed output tiles. It explains how scale ownership and circular-stage completion constrain the arithmetic after the scheduler and cluster have assigned the work.

Table 23.16.: Concrete SM90 swizzle coordinates. The retained rows use the 32×11 grid and $G = 16$. The constructed five-by-two grid shows why an odd multicast tail is split into a one-wide unicast group.

Case	Logical index t	Swizzled tile (m, n)	Consequence
Retained slot 17	17	(1, 1)	first claimed tile
Retained slot 17	149	(5, 9)	same 16-block group
Retained slot 17	281	(25, 6)	second 16-block group
Retained slot 100	100	(4, 6)	first claimed tile
Retained slot 100	232	(24, 3)	second 16-block group
Five-by-two tail	8	(4, 0)	unmatched primary tile; unicast
Five-by-two tail	9	(4, 1)	unmatched primary tile; unicast

Plan choice	Intended benefit	Preserved invariant	Limiting condition or evidence
Persistent claiming	Reuse a bounded set of physical CTA slots	Every logical tile has one output owner	Tail claims stop outside the logical tile set.
Schedule swizzle	Improve operand reuse or distribute work	Tile coordinates and results do not change	The ordering must remain a one-to-one tile mapping.
Two-CTA multicast	Avoid one duplicate <i>A</i> - or <i>B</i> -tile transfer	Each CTA keeps its output and nonshared operand	Cluster legality and edge peers can require unicast.
Tile and stage count	Trade reuse and parallelism against TMA latency	Full/empty completion still guards every stage	Registers, shared memory, occupancy, and architecture constrain candidates.
Compiled resource check	Confirm registers, local memory, and the emitted artifact	A source plan is not itself compiled-path evidence	PTXAS and cubin evidence must match the selected specialization.

Table 23.17.: Persistent work claiming and multicast solve different scheduling problems. Both remain bounded by output ownership, tail legality, resource limits, and evidence from the selected artifact.

Example 23.7.2 (A scale seam inside one generated output tile) Choose output tile $q = 1$, whose N -coordinates are $j = 192, \dots, 383$. The selected tile width is 192, but the SM90 weight-scale recipe owns 128×128 blocks. The tile therefore contains two scale regions:

$$\underbrace{j = 192, \dots, 255}_{64 \text{ columns using } s_{1,r}^B} \quad | \quad \underbrace{j = 256, \dots, 383}_{128 \text{ columns using } s_{2,r}^B}.$$

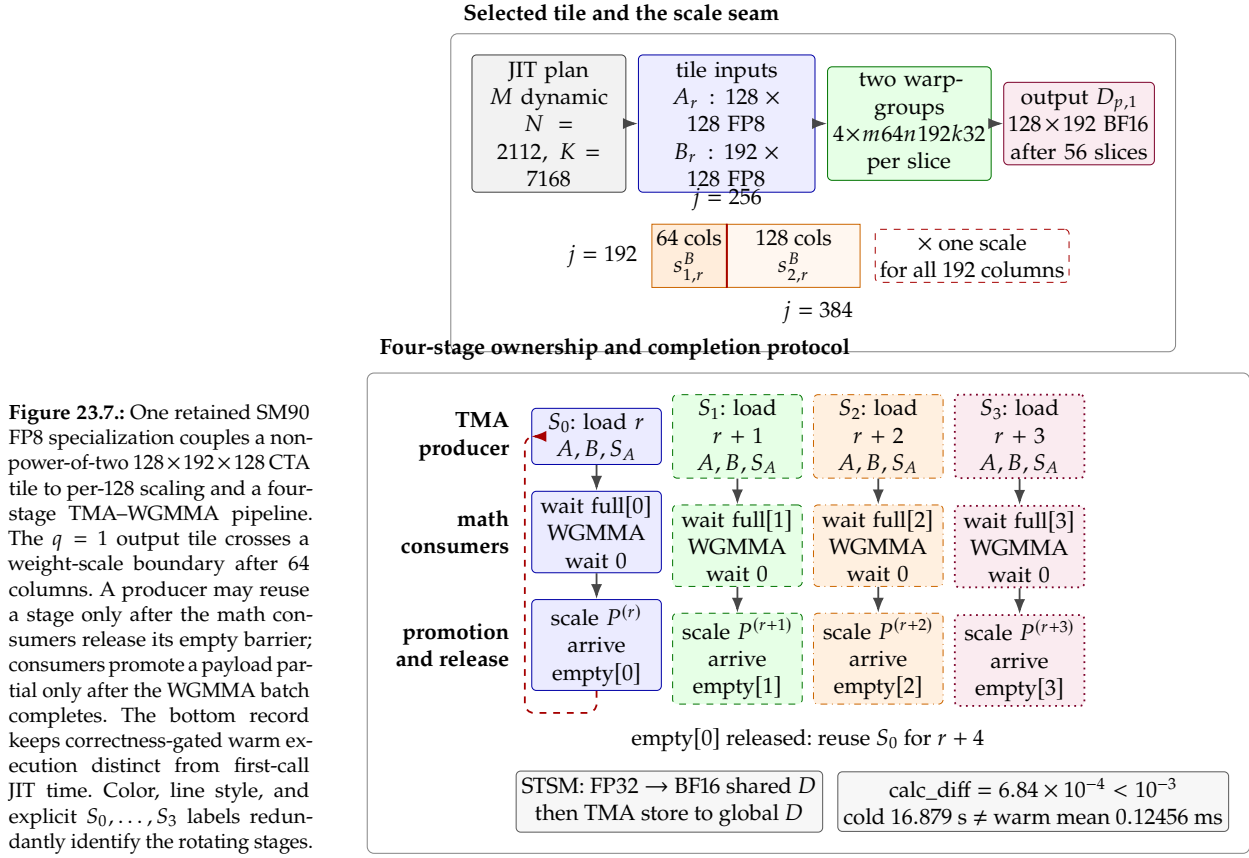
For K -block $r \in \{0, \dots, 55\}$, define the unscaled payload partial and the promoted accumulator by

$$P_{ij}^{(r)} = \sum_{\ell=0}^{127} q_{i,128r+\ell}^A q_{j,128r+\ell}^B, \quad F_{ij}^{(r+1)} = F_{ij}^{(r)} + s_{i,r}^A s_{\lfloor j/128 \rfloor, r}^B P_{ij}^{(r)}.$$

Two math warpgroups cover the upper and lower 64 rows. Within each $K = 128$ slice, each warpgroup issues four *m64n192k32* WGMMA operations, waits for their completion, and only then promotes the payload partial with the matching scales. Using one B scale across all 192 columns would preserve the payload matrix shapes while changing the represented product. Figure 23.7 locates this scale seam inside the generated pipeline and its staged-storage accounting.

One pipeline stage holds 16 KiB of A payload, 24 KiB of B payload, and 512 bytes of A scales. The producer therefore attaches 41,472 expected transaction bytes to that stage's full barrier. Four such stage allocations plus the 48-KiB shared output tile occupy 215,040 bytes before the separately loaded B -scale buffer and barrier objects. The pipeline visual below joins that storage accounting to the scale seam and the full/empty reuse protocol.

The figure exposes three independent failure classes. Ignoring the 64|128 scale seam changes represented values. Reusing S_0 before empty[0] permits the producer to overwrite shared operands still owned by a consumer. Promoting before



the WGMMA wait consumes an incomplete payload partial. None of those kernel errors is repaired by a passing cache lookup, and none makes the 16.879-second first call comparable to the 0.12456-millisecond warm mean.

The JIT lifecycle ends with a selected runtime artifact and a recorded cold or warm event. Chapter 35 reports the retained H200 specialization, its TMA/WGMMA pipeline, resource use, and cold/warm timings.

23.8. Grouped GEMM for Routed Experts

The dense running example used six token rows. In an MoE layer, a router can turn those six rows into more work because a token can be sent to multiple experts. Earlier MoE chapters introduced sparse active computation and expert routing. Here the local question is narrower: once routed rows exist, how can their expert-specific matrix products be expressed as a grouped GEMM call? DeepGEMM calls this M -axis surface M -grouped GEMM: grouping occurs along the M axis because experts share N and K while their valid row counts M_e vary with the numbers of routed tokens they receive [81].

Definition 23.8.1 (Routed expert row) *A routed expert row is one activation row paired with one selected expert. If top- k_{top} routing sends each token to k_{top} experts, then M token rows can produce up to Mk_{top} routed expert rows before masking or capacity limits.*

In the MoE running example, six tokens use top-2 routing. The twelve routed rows are distributed across three experts with counts

$$(M_0, M_1, M_2) = (5, 4, 3).$$

These six tokens are a local batching fixture. The single REQUEST-TRACE/SCALED row ρ_7 can instantiate one valid grouped row, but it does not determine the fixture's other eleven routes or turn the grouped call into measured execution. Each expert has a same-shape weight matrix $B_e \in \mathbb{R}^{12 \times 8}$, and the routed activation block for expert e is $A_e \in \mathbb{R}^{M_e \times 8}$. The grouped mathematical work is still three ordinary GEMMs:

$$D_0 = A_0 B_0^\top, \quad D_1 = A_1 B_1^\top, \quad D_2 = A_2 B_2^\top.$$

The checked M-grouped wrappers write D_e directly and do not accept an additive C_e input; relative to the chapter's dense baseline, they implement the case $C_e = 0$ [81]. The grouped interface is a way to present those related products to a kernel family, not a new mathematical product.

Definition 23.8.2 (M-grouped GEMM) *A grouped GEMM is a collection of GEMM instances executed through one grouped interface. In the M-grouped form emphasized here, group e computes $D_e = A_e B_e^\top$, the experts share N and K , and the row counts M_e may vary by expert. The M in the name identifies the dimension whose valid extent changes across groups.*

For the M-grouped GEMM in Definition 23.8.2, DeepGEMM exposes two physical layouts. In a contiguous layout, valid routed rows are concatenated into one logical $M_{\text{total}} \times K$ input, and a layout vector records how the rows are split by expert. With counts (5, 4, 3), the unpadded segment boundaries are 5, 9, and 12. If a toy alignment of four rows is used, the first expert segment is padded to 8 rows, the second remains 4, and the third is padded to 4, for 16 physical row slots. The padding is launch structure, not mathematical work.

Definition 23.8.3 (Contiguous and masked grouped layouts) *A contiguous grouped layout concatenates group rows and records segment boundaries or aligned lengths. A masked grouped layout reserves a per-group capacity and supplies a valid row count for each group. In both cases, invalid or padded rows must not contribute to the mathematical output.*

Grouped GEMM batches ordinary expert GEMMs; padding and masks are launch layout, not extra routed tokens.

Algorithm 7: Pack and execute M-axis grouped expert GEMM without inventing routed rows.

Input: Retained route records from the route–dispatch–combine procedure in the *DeepSeekMoE* and *Sparse Computation* chapter in Volume I; expert matrices W_e ; contiguous or masked layout L ; alignment/capacity metadata

Output: Expert results tagged by the grouped row-order record

```

1 initialize one physical group buffer and valid-row count  $M_e \leftarrow 0$  per expert;
2 foreach retained route record  $(i, e, r, w, x_i)$  in stable route order do
3   choose the next valid physical slot  $s \leftarrow \text{slot}(L, e, M_e)$ ;
4   if  $s$  exceeds the declared segment or capacity then
5     return overflow under the declared capacity/no-drop policy;
6   store  $x_i$  in slot  $s$ , attach  $(i, e, r, w)$ , and increment  $M_e$ ;
7 fill alignment or unused-capacity slots without attaching row-order records;
8 foreach expert  $e$  do
9   compute  $Y_e[0:M_e] \leftarrow X_e[0:M_e]W_e^T$ ;
10 foreach valid physical output slot do
11   emit its expert result with the attached token, expert, route-rank, and weight record;
12 return tagged results for the weighted token-order combine;

```

Definition 23.8.4 (Grouped row-order record) *A grouped row-order record maps each valid physical row slot to the token, expert, and route rank that produced it. It tells the combine step where the expert result returns in token order. Contiguous layouts can encode this through segment order plus the routed-row list; masked layouts can encode it through per-expert capacity, valid counts, and valid-slot order. Padded or masked-out slots have no row-order record.*

Definitions 23.8.3 and 23.8.4 separate physical capacity from the identity map that combine needs.

Table 23.18.: A row-order record keeps routed token semantics stable across contiguous and masked grouped layouts.

Routed pair	Contiguous interpretation	Masked interpretation	Combine obligation
(t_1, e_0, r_1)	expert 0 segment, valid row 1	expert 0 capacity, valid slot 1	return to token t_1 , route r_1
(t_1, e_2, r_2)	expert 2 segment, valid row 1	expert 2 capacity, valid slot 1	return to token t_1 , route r_2
(t_2, e_1, r_1)	expert 1 segment, valid row 1	expert 1 capacity, valid slot 1	return to token t_2 , route r_1

A small route trace in Table 23.18 makes the row-order idea concrete. The same routed pair may occupy a different physical slot under a contiguous or masked layout, but the combine obligation is unchanged.

Algorithm 7 begins after routing: it does not choose experts again. First, it places every retained route in a physical slot and attaches the identity needed later. Then it pads or reserves capacity without creating semantic rows and computes only each expert’s valid prefix. Finally, it emits results with the same identity record for weighted combine. Thus contiguous and masked layouts may change physical placement without changing route multiplicity, token order, or the no-drop/capacity decision.

The padding rule becomes concrete in Table 23.19. The physical row slots can exceed the valid routed rows, but the mathematical output for the example is still defined by $5 + 4 + 3 = 12$ routed rows. A masked layout reserves a common

Expert	Valid rows M_e	Unpadded end	Toy padded rows	Toy padded end
0	5	5	8	8
1	4	9	4	12
2	3	12	4	16

Table 23.19.: Contiguous grouped layout for the routed running example with a toy four-row alignment.

capacity instead. With capacity six, its input payload has physical shape $3 \times 6 \times 8$, while the valid-count vector (5, 4, 3) limits the three expert computations. This stable-capacity form is useful during inference decoding when CUDA graph use means the CPU may not know each expert’s token count; DeepGEMM names low-latency DeepEP output as one example input surface [81, 19]. Figure 23.8 compares the two physical encodings of the same valid routed rows.

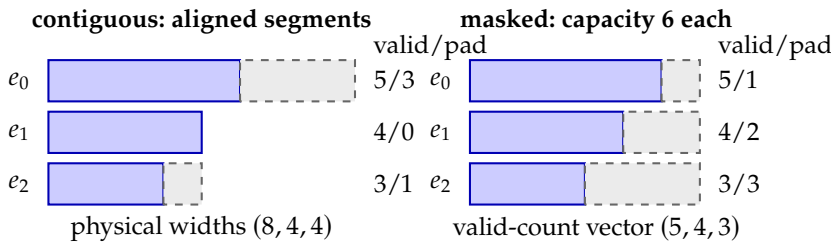


Figure 23.8.: Contiguous and masked M-grouped layouts encode the same expert counts (5, 4, 3) differently. Contiguous segments have individually aligned physical widths; masked groups reserve six slots each and carry valid counts. Solid colored spans are valid rows, while dashed gray spans are padding or unused capacity; neither kind of invalid slot contributes to the mathematical output.

The raster flow in Figure 23.9 depicts the same MoE compute boundary: routed rows are packed by expert, grouped GEMM runs over the packed segments, and the row-order record remains needed afterward.

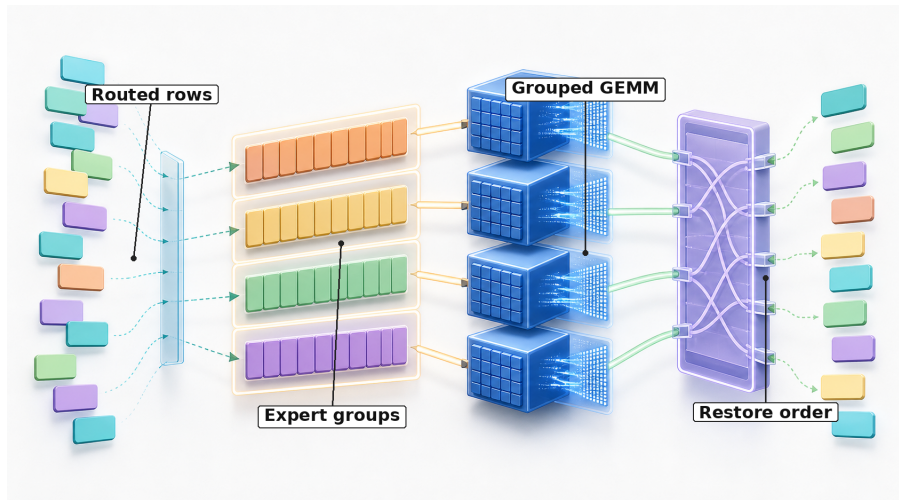


Figure 23.9.: Grouped expert GEMM packs routed rows into expert groups for compute while preserving the row-order information needed by combine.

The M-axis MoE layout is not the only grouped surface in the checked API. For MoE weight backward, each expert produces one fixed-shape weight-gradient matrix but reduces over the routed rows assigned to it. When K-group g represents expert e , the forward count M_e becomes the backward reduction extent K_g , while the output dimensions remain fixed. Padding every group to the largest reduction width would move and multiply non-semantic coordinates. DeepGEMM instead exposes K-axis grouped contiguous GEMM, which concatenates the real reduction slices and retains their group boundaries [81].

Definition 23.8.5 (K-axis grouped contiguous GEMM) *A K-axis grouped contiguous GEMM holds M and N fixed and assigns group g a reduction extent K_g . In the TN teaching orientation, the passed slices satisfy*

$$A_g \in \mathbb{R}^{K_g \times M}, \quad B_g \in \mathbb{R}^{K_g \times N}, \quad D_g = C_g + A_g^\top B_g \in \mathbb{R}^{M \times N}.$$

Here M is the first fixed output dimension of the TN-oriented backward product, not the forward routed-row count M_e . The concatenated A and B buffers have leading extent $\sum_g K_g$, while $C, D \in \mathbb{R}^{G \times M \times N}$ keep group outputs separate. Aligned gaps or tails are physical layout, not extra reduction terms.

Definition 23.8.5 changes the grouped axis from routed rows to reduction slices while preserving one result per group.

This definition uses the TN orientation exposed by the BF16 and SM100 FP8 wrappers. The pinned SM90 low-precision surface names its eligible layout `k_grouped_fp8_gemm_nt_contiguous`; SM100 names `k_grouped_fp8_gemm_tn_contiguous`, and BF16 uses `k_grouped_bf16_gemm_tn_contiguous` on both architectures. The mathematical group result is unchanged by those passed-layout names. In the ordinary contiguous form, the group record supplies aligned K_g lengths. The optional SM100 partial-sum layout instead supplies group end offsets in an aligned buffer. In either form, the pinned repository tests use FP32 C as the in-place D buffer and compare every group with its own reference product [81].

Definition 23.8.6 (Partial-sum end-offset layout) *Let e_g be the exclusive physical end offset recorded for group g , and let A be the required alignment. Group zero starts at $s_0 = 0$; each later group starts at*

$$s_g = \text{align}(e_{g-1}, A).$$

The valid coordinates of group g are $[s_g, e_g)$. Any coordinates in $[e_{g-1}, s_g)$ are alignment gaps, not semantic rows or reduction terms. For M -grouped partial sums these offsets delimit routed rows; for K -grouped partial sums they delimit reduction slices [81].

For a K-axis teaching trace with alignment $A = 4$ and end offsets $(e_0, e_1) = (3, 9)$, group zero owns $[0, 3)$, coordinate 3 is a gap, and group one owns $[4, 9)$. Thus $(K_0, K_1) = (3, 5)$; treating the record as ordinary lengths $(3, 9)$ would consume the gap and overrun the second group. The UE8M0 layout kernels honor the same boundary. On an M-axis partial-sum layout, the transpose-and-pack kernel writes the finite exponent code `00` for gap rows instead of reading their uninitialized FP32 scale slots; it avoids the UE8M0 NaN code `ff`. On a K-axis partial-sum layout, the packer skips the physical gaps and emits compact per-group scale rows. In both cases, gap scale storage makes device access safe but never creates a valid payload coordinate [81].

Example 23.8.1 (Two K-groups remain two outputs) Take $G = 2$, $M = N = 1$, $K_0 = 2$, and $K_1 = 1$. Ignore physical alignment for this logical trace and concatenate

$$A = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad B = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}, \quad C_0 = [10], \quad C_1 = [20].$$

The group record $(2, 1)$ assigns the first two reduction rows to group zero and the last row to group one. Therefore

$$D_0 = 10 + [1, 2] \begin{bmatrix} 4 \\ 5 \end{bmatrix} = 24, \quad D_1 = 20 + [3][6] = 38.$$

The kernel returns the two 1×1 group outputs $[24]$ and $[38]$. Summing them would change the declared output; consuming padded coordinates would change one of the declared reductions. Figure 23.10 places this reduction-axis trace beside the earlier routed-row path.

M-grouped forward work varies output-row ownership, whereas K-grouped backward-style work varies the reduction slice and keeps one $M \times N$ output per group.

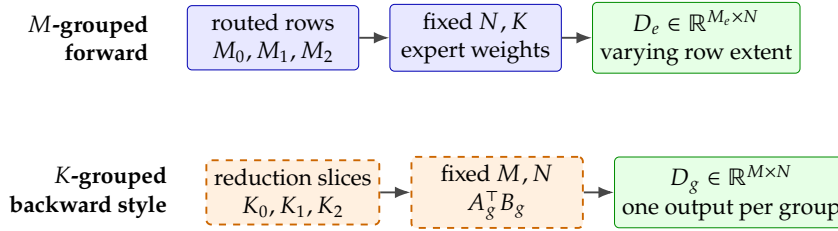


Figure 23.10.: “Grouped” names two different varying axes. M-grouped GEMM varies valid output rows M_e while sharing N, K ; K-grouped GEMM varies reduction extents K_g while sharing M, N and preserving one output matrix per group. Color, border style, and row labels distinguish the two paths.

The correctness rules are simple enough to state and strong enough to catch bugs. For every valid M-grouped routed row, the grouped output must equal the corresponding expert’s ordinary GEMM output under the same dtype and scale configuration; padded or masked-out rows create no semantic token contribution. For every K-grouped output D_g , only the real reduction coordinates owned by group g may contribute, and the result must remain in that group’s output slot. Later systems material can explain dispatch and load balancing. The local obligation is to make the compute semantics precise.

An open runtime provides one source-backed check that these compute semantics have a concrete consumer. At the recorded SGLang source snapshot, the grouped MoE runner carries hidden states, row indices, masks, valid-row counts, expert weights, and FP4/FP8 scale metadata into contiguous or masked DeepGEMM paths [72]. This confirms that the chapter’s fields cross a real integration handoff; it does not establish that every request selects the backend. A runtime claim must still record the backend-selection branch, artifact revision, grouped layout, scale ownership, numerical oracle, and timing boundary.

Grouped GEMM ends at the local expert-output contract. The downstream Chapter 27 begins when dispatch, activation and route weighting, both expert

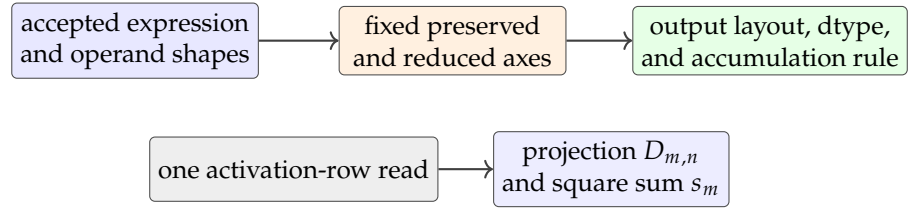
linears, remote contribution write-back, and combine become one persistent multi-rank execution interval.

23.9. Einsum and HyperConnection Prenorm

Not every transformer contraction arrives at the kernel boundary as one two-dimensional $M \times K$ by $N \times K$ product. Attention heads, residual streams, and parameter-gradient reductions retain batch or head axes whose ownership matters. Flattening those axes can duplicate work, require an extra transpose, or lose an accumulation rule. DeepGEMM therefore exposes a small, explicit set of Einstein-summation kernels rather than claiming a general einsum compiler [81].

Figure 23.11 separates the two finite surfaces developed in this section.

Figure 23.11.: Two finite kernel interfaces. An einsum expression fixes axis and output semantics; HyperConnection prenorm shares one activation read across the projection and row-square reductions.



Definition 23.9.1 (Released DeepGEMM einsum surface) *The released einsum surface is the set of exact expression strings accepted by `einsum` or `fp8_einsum`, together with their axis shapes, payload and accumulator dtypes, scale metadata, architecture gates, optional accumulation tensor, and output layout. An algebraically equivalent expression outside that set is not automatically a valid DeepGEMM call.*

The BF16 entry point recognizes three contractions:

$$D_{m,n} = C_{m,n} + \sum_{b,k} A_{b,m,k} B_{b,n,k}, \quad \text{bmk, bnk} \rightarrow \text{mn}, \quad (23.1)$$

$$D_{b,h,d} = \sum_r A_{b,h,r} B_{h,d,r}, \quad \text{bhr, hdr} \rightarrow \text{bhd}, \quad (23.2)$$

$$D_{b,h,r} = \sum_d A_{b,h,d} B_{h,d,r}, \quad \text{bhd, hdr} \rightarrow \text{bhr}. \quad (23.3)$$

`bmk, bnk` $\rightarrow$ `mn` reduces the b axis; the two head-wise contractions preserve both b and h .

The first contraction reduces the leading batch axis into one m, n output. When that output is FP32, the pinned implementation requires C to alias D : the call accumulates in place. When BF16 output is requested, the wrapper allocates an FP32 workspace, accumulates there, and copies the result to BF16. The two head-wise contractions preserve both b and h , and their output has no optional C at this surface.

Example 23.9.1 (Why the batch reduction is not an ordinary BMM return) Let $b = 2, m = n = 1, k = 2$,

$$A_{0,0,:} = [1, 2], \quad B_{0,0,:} = [3, 4], \quad A_{1,0,:} = [2, 0], \quad B_{1,0,:} = [5, 6].$$

Equation 23.1 returns the single value

$$(1 \cdot 3 + 2 \cdot 4) + (2 \cdot 5 + 0 \cdot 6) = 21.$$

A batched matrix product would instead retain two batch outputs, 11 and 10. The expression string therefore fixes an ownership and reduction rule, not just a convenient spelling of GEMM.

The CUDA implementation makes that reduction concurrent. It flattens the $(b, K/\text{BLOCK_K})$ work, chooses a split factor from the number of available SMs and output tiles, and lets multiple CTAs contribute partials to the same FP32 (m, n) tile. In the logical example above, one split can be read as contributing 11 and another as contributing 10; the stored answer is their reduction, 21, not either partial. SM90 commits those FP32 partials with vectorized atomic adds, while SM100 uses a TMA reduce-add store. For BF16 output, the wrapper first zeroes an FP32 workspace, completes the same split reduction there, and casts only the finished result to BF16 [81].

Invariant 23.9.2 (Every batch-reduction split reaches the FP32 destination). *For $\text{bmk}, \text{bnk} \rightarrow \text{mn}$, every split of the flattened batch- K domain must contribute exactly once to the shared FP32 output tile before a BF16 conversion or consumer observes it. A launch that returns a per-split tile, omits one reduce-add, or casts each partial separately implements a different numerical contract.*

Invariant 23.9.2 is the oracle for every supported split factor before the output is converted or consumed. The low-precision `fp8_einsum` path accepts a smaller architecture-dependent expression set. It implements `bhr, hdr → bhd` on SM90 and SM100. SM100 additionally implements `bhd, hdr → bhr` and the parameter-gradient-style contraction

$$D_{h,d,r} = C_{h,d,r} + \sum_b A_{b,h,d} B_{b,h,r}, \quad \text{bhd, bhr} \rightarrow \text{hdr}.$$

The FP8 operands travel with scale tensors whose axes are permuted with the payload before the batched GEMM dispatch. Equation 23.9 uses FP32 in-place accumulation and an SM100-only orientation. This chapter does not generalize that support to arbitrary Einstein expressions.

Table 23.20 makes the retained reduction axes, dtype paths, and architecture gates explicit.

Fusing the two HyperConnection prenorm reductions HyperConnection maintains multiple residual streams, so a prenorm transition needs both a small projection of each activation row and the row energy used by normalization. DeepGEMM does not implement the full residual-stream transition; it fuses

Table 23.20.: DeepGEMM’s ein-sum API is a finite set of supported axis patterns, not an arbitrary expression engine.

Expression	Output ownership	Released dtype/architecture	Required oracle
<code>bnk, bnk->mn</code>	Reduce batch b ; one m, n matrix	BF16 inputs, SM90/SM100; FP32 in-place accumulation or BF16 output through FP32 workspace	Sum of per-batch FP32 products plus optional aliased C .
<code>bhr, hdr->bhd</code>	Preserve batch and head	BF16 on SM90/SM100; FP8 on SM90/SM100	Per-head contraction with payload/scale axes aligned.
<code>bhd, hdr->bhr</code>	Preserve batch and head	BF16 on SM90/SM100; FP8 on SM100	Contract d , preserve r , and compare every head slice.
<code>bhd, bhr->hdr</code>	Reduce batch into a parameter-shaped result	FP8 on SM100 with FP32 accumulation	Quantized simulated reference plus initial FP32 C , with the batch reduction explicit.

these two inputs because both reduce the same activation matrix A . Given $A \in \text{BF16}^{M \times K}$ and $B \in \text{FP32}^{N \times K}$, one traversal of A produces

$$D_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{n,k}, \quad (23.4)$$

$$s_m = \sum_{k=0}^{K-1} A_{m,k}^2. \quad (23.5)$$

The first output supplies a small FP32 projection used by the HyperConnection prenorm path; the second supplies the rowwise square sum needed for normalization. Computing them separately would reread the same activation row.

The exported `tf32_hc_prenorm_gemm` requires A and B to be K -major, D to be N -major, and s to be contiguous. Despite its name, the activation input is BF16 and the projection matrix, product output, and square-sum output are FP32. Both SM90 and SM100 have released implementations. Both require K to be a multiple of 64 and N to be a positive multiple of 8. SM90 permits $N \leq 32$, while SM100 permits $N \leq 128$. An empty M is a no-op; K must be positive [81].

For a split launch with $P = \text{num_splits}$, the call does not directly return final D and s . It writes

$$D^{(p)} \in \mathbb{R}^{M \times N}, \quad s^{(p)} \in \mathbb{R}^M, \quad p = 0, \dots, P-1,$$

The consumer must therefore form the following two reductions across all P split outputs:

$$D = \sum_{p=0}^{P-1} D^{(p)}, \quad s = \sum_{p=0}^{P-1} s^{(p)}.$$

Using only split zero or averaging the splits violates both outputs. The repository test covers both unsplit output shapes and a 16-split case, then compares the two reduced outputs with Equations 23.4 and 23.5.

Example 23.9.2 (One row, two prenorn outputs) For $A = [1, 2, 2]$ and

$$B = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix},$$

the fused kernel returns $D = [3, 4]$ and $s = 9$. If two splits own coordinates $\{0, 1\}$ and $\{2\}$, valid partials are $D^{(0)} = [1, 2]$, $D^{(1)} = [2, 2]$, $s^{(0)} = 5$, and $s^{(1)} = 4$. Only the sums reproduce the unsplit result.

Chapter 24 develops the surrounding mHC state transition: its pre-mix and post-mix equations preserve multiple residual streams, while its recomputation invariant preserves BF16 store/reload points. The DeepGEMM kernel here is narrower. It produces the FP32 projection and row square sum that a prenorn stage can consume; it does not implement the complete mHC transition, Sinkhorn normalization, or autograd ownership described there.

Invariant 23.9.3 (Both prenorn outputs share one reduction domain). *The product D and square sum s must use the same complete K -domain and, when split, the same complete set of split contributions. A correctness record compares both reduced outputs; agreement of only D cannot validate the normalization statistic.*

Invariant 23.9.3 fixes the paired-output oracle for this surface. No local timing or tolerance row has been run for these einsum and HyperConnection surfaces in this chapter. They are included because they are released, tested repository interfaces at the pinned commit. Any later performance claim must add architecture, exact expression or split count, layout, dtype and scale state, JIT controls, and the corresponding reference comparison.

23.10. Benchmarking, Numerical Tolerance, and Reproducibility

Numerical tolerance policy. DeepGEMM changes both arithmetic and data representation, so performance and correctness must be interpreted together. A fast FP8 or FP4 path is useful only if its scales, grouped layout, masks, and accumulation policy reconstruct the intended result. The relevant comparison is therefore not “kernel versus kernel” in the abstract, but one fixed shape and layout against a named reference under a declared numerical metric.

A low-precision speed result matters only when the same shape and dtype path also pass a stated tolerance.

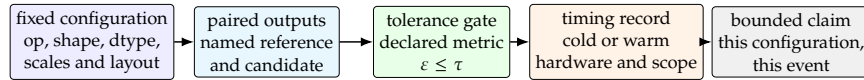
Validation Rule 23.10.1 (Comparable timing identity). A direct speed comparison holds the logical operator, shape, payload dtype, scale recipe, operand and grouped-work layouts, architecture path, and timing protocol fixed. If one of those fields changes, the rows describe different kernel configurations; a speed ranking requires a matched comparison.

Reported release calibration. REPORTED At the chapter’s H200 measurement pin, the DeepGEMM release reports up to 1550 TFLOP/s on H800. The README headline links the optimization changes but omits a fully stated shape and timing protocol for that maximum. It therefore provides release-level calibration. The book’s fixed-shape H200 specialization supplies its own baseline, and a Hopper-wide guarantee would require broader shape coverage [81].

Definition 23.10.2 (Numerical tolerance policy) *Chapter 32, introduced later, develops the general view of a tolerance as an acceptance region with named coordinates. A DeepGEMM policy binds that region to the reference implementation, compared tensors, dtype path, shape, fixed inputs or seed, error metric, and threshold. The policy belongs to the comparison record: changing the scale recipe, grouped layout, mask, or accumulator can require a different threshold.*

Definition 23.10.2 binds the acceptance region to one configuration. Figure 23.12 makes the evidence gate explicit. Correctness and timing attach to the same fixed configuration: the candidate must pass its declared comparison before the timing record supports a bounded performance claim.

Figure 23.12.: A DeepGEMM timing supports a bounded performance claim only when the same fixed configuration produces a candidate that passes its declared numerical tolerance and the measured event is identified. Comparisons across shapes, layouts, or dtype paths require additional matched rows.



Maximum absolute error is simple but can hide scale differences across tensors. Relative error needs a denominator convention because values near zero can dominate it. DeepGEMM’s test utilities also use

$$1 - \frac{2\langle x, y \rangle}{\|x\|^2 + \|y\|^2},$$

with a zero-vector special case [81]. For $x = (1, 2)$ and $y = (1.1, 1.9)$, this metric is approximately 0.0020. It summarizes directional and magnitude agreement, but its threshold still belongs to a particular dtype path and test shape.

Systems evidence and integration. One representative H200 specialization closes the chapter’s own timing argument. For $(M, N, K) = (4096, 2112, 7168)$ on the SM90 FP8 path, the BF16 output passed the recorded comparison with $\text{calc_diff} = 6.84 \times 10^{-4} < 10^{-3}$. Five repeat means after five warmups gave a 0.12456 ms mean and 0.11403 ms median; the mean corresponds to approximately 996 semantic TFLOP/s under the $2MNK$ convention. These values describe one generated plan on one H200. The source-reported H800 maximum and shape-family rankings have different scopes.

The broader sampled H200 sweep pairs the same utility metric with other shapes and layouts. Its complete timing matrix, first-call boundary, retained generated artifacts, and closeout states remain in Chapter 35. The interpretive rule remains

here: a timing is admissible only after its declared comparison passes. A path ranking requires matched shapes, layouts, scale recipes, and protocols.

Runtime integration adds another distinction. SGLang contains direct DeepGEMM wrapper and MoE-runner surfaces alongside other quantized kernels [72]. A serving result should credit DeepGEMM only when the selected path reaches that wrapper, and it should separate compilation or cache loading from steady-state execution. The retained one-H200 integration joins runtime selection, wrapper entry, and launched SM90 kernels; its exact counts and boundary are consolidated beside FlashMLA in that execution dossier. Its scope is generated-kernel execution rather than request latency or another MoE backend.

SM120 and SM110 runs reached recorded architecture, recipe, or MQA guards without emitting an admitted timing row. Those diagnostics are neither numerical failures nor evidence about adjacent architecture families. The operator-specific 192-case SM90 MQA correctness matrix remains local because it teaches DeepGEMM's selection and oracle surface rather than a cross-system performance comparison.

Compact reproducibility check. For a small reproducible check, fix $A \in \mathbb{R}^{6 \times 8}$, $B \in \mathbb{R}^{12 \times 8}$, and an optional $C \in \mathbb{R}^{6 \times 12}$. Compute a BF16 reference, run the target kernel, and report the exact shape, dtype and scale path, chosen metric, and threshold. Grouped MoE cases must additionally preserve route counts, padding or mask policy, and per-group reference slices. This compact protocol is enough to interpret a result without turning the chapter into a catalogue of run artifacts.

23.11. MQA and Indexer-Logit Kernels

DeepGEMM also contains an adjacent attention-side kernel family. Its FP8 or packed-FP4 payloads, scale metadata, TMA descriptors, and JIT machinery resemble GEMM, but its output is an attention or indexing score over candidate cache positions rather than an $M \times N$ result. Separating this family prevents dense or grouped-GEMM evidence from being misattributed to an attention-indexer result.

Definition 23.11.1 (FP8/FP4 MQA indexer-logit kernel) *An FP8/FP4 MQA indexer-logit kernel takes quantized query vectors, quantized single-KV-head keys, scale metadata, and optional per-head weights, then writes logit or index-score records for candidate cache positions. Dense-sequence variants use cumulative key-start and key-end metadata; paged variants add context lengths, block tables, optional selected indices, and scheduling metadata. At the pinned revision SM90 exposes FP8, while SM100 also exposes packed FP4 query and KV records with packed UE8M0 scales. The correctness target is therefore the attention-indexing score under its metadata-aware oracle, not a dense or grouped GEMM output.*

The V3.2-style reference makes the distinction visible. Its public inference kernel loads FP8 q and FP8 k , multiplies them in FP32, clamps negative per-head logits to zero, applies query scales, reduces over heads, and applies key scales to produce an index score [20]. The DeepGEMM path optimizes the same class of score production with SM90 FP8 and SM100 FP8/FP4 dense and paged MQA

These kernels produce candidate-position scores; query/KV scales, validity metadata, and output coordinates belong to their oracle.

logit kernels. The dense wrapper specializes head count, head dimension, query and KV tile sizes, compressed-logit mode, shared-memory stages, and output dtype. The paged wrapper adds block-table stride, context-length format, selected indices, schedule metadata, and split-KV tiling [81]. Table 23.21 records the state required to interpret and validate this kernel family.

Table 23.21: Coverage fields for DeepGEMM FP8/FP4 MQA indexer-logit kernels.

Field	Coverage obligation for the kernel family
Quantized q , k , and scales	The score is not interpretable without the FP8 or packed-FP4 payload dtype, query scales, key or KV-cache scales, and accumulator convention.
Per-head weights and reduction	MQA/indexer logits may weight or reduce headwise scores; the weights and reduction belong in the oracle.
Dense sequence metadata	Cumulative key-start and key-end arrays define which cache positions are valid for each query span.
Paged cache metadata	Context lengths, block tables, optional selected indices, and schedule metadata define the physical cache pages that represent the logical candidate positions.
JIT specialization	Head dimension, tile sizes, split-KV factor, compressed-logit mode, output dtype, and SM90 shared-memory budget are part of the selected kernel record.

A reviewed single-H200 rerun at the measured DeepGEMM commit 54e226124093 completed the upstream SM90 dense-prefill enumerator. It requested, selected, found available, and observed all 192 cases. Table 23.22 records this as direct-kernel capability evidence rather than paged, FP4, or serving coverage.

These kernels support FP8 and, on SM100, FP4 index-score production; they do not establish grouped-MoE throughput. Conversely, dense GEMM tolerances do not validate paged-cache index scores without the cache metadata and score oracle in Definition 23.11.1.

The SM110 diagnostic selected one of the 192 upstream dense-prefill cases on each sampled Jetson Thor device. Every subprocess reached the pinned wrapper predicate that admits architecture major 10 with 8/16/32/64 heads or major 9 with 32/64 heads; architecture major 11 was rejected. Thus three cases were selected and zero metric rows were observed. This exact guard is evidence about the recorded wrapper route, not a numerical MQA failure or a statement about every possible SM110 implementation.

Released dense and paged indexer interfaces Three exported operations define the modern surface. The dense operation is `fp8_fp4_mqa_logits`. The paged pair separates metadata construction from `fp8_fp4_paged_mqa_logits` execution; the FP8-only names remain legacy wrappers. For dense prefill, each query row i owns a logical interval $[s_i, e_i)$. For candidate j in that interval, the reference computes

$$r_{i,h,j} = \max(0, \langle q_{i,h}, k_j \rangle), \quad z_{i,j} = \sum_h w_{i,h} r_{i,h,j}.$$

The kernel may return a full token-by-token logit matrix and clean unfilled coordinates to $-\infty$, or return a compressed row whose width is the maximum $e_i - s_i$. Those modes have different output coordinate systems, so the

Field	Recorded H200 evidence
Dense-prefill matrix	FP8 only; BF16 or FP32 logits; FP32 reduction; full logits with cleaning or compressed logits without cleaning; $S_Q \in \{2048, 8192\}$, $S_{KV} \in \{8192, 65536\}$, $H \in \{32, 64\}$, $D \in \{32, 64, 128\}$, and context-parallel metadata disabled or enabled. These axes form 192 unique tuples.
Coverage check	Requested, selected, available, and observed counts were all 192; <code>complete_for_selected_cases</code> was true and the subprocess returned zero.
Correctness gate	Every case passed exact negative-infinity-mask equality, twenty repeated bitwise-consistency checks, reference difference below 10^{-3} , and simulated-quantized-reference difference below 3×10^{-5} for BF16 logits or 5×10^{-6} for FP32 logits. Python and PyTorch seeds were both zero; the individual difference values were not serialized.
Representative profiler row	For FP32 full logits with cleaning, $S_Q = 2048$, $S_{KV} = 8192$, $H = 32$, $D = 64$, and context-parallel metadata enabled: 61 μs , 564 TFLOP/s, and 634 GB/s for the score kernel; 17 μs and 1989 GB/s for cleaning. This one row is not an across-shape ranking.
Follow-up boundary	A later four-case packet adds serialized dense and paged differences plus automatic page-size fallback. A model-backed V3.2 lightning-indexer serving route was not measured because the official 671B artifact exceeds four H200s and V2-Lite has no indexer.

Table 23.22.: Recorded SM90 FP8 dense-prefill MQA-logit cases, with one representative profiler row. The result does not establish paged, FP4, or serving behavior.

oracle must expand compressed rows back to their logical intervals before comparing masks or scores [81].

Example 23.11.1 (One weighted-ReLU index score) Take one query row, two heads, head dimension two, and unit payload scales, so the represented vectors are

$$q_{i,0} = [1, -1], \quad q_{i,1} = [2, 1], \quad k_0 = [1, 2], \quad k_1 = [1, -2].$$

Let the head weights be $w_i = [\frac{1}{4}, \frac{3}{4}]$ and the valid dense interval be $[s_i, e_i) = [0, 2)$. For candidate zero, the two raw dot products are -1 and 4 ; after ReLU and weighting,

$$z_{i,0} = \frac{1}{4} \max(0, -1) + \frac{3}{4} \max(0, 4) = 3.$$

For candidate one, the raw scores are 3 and 0 , so

$$z_{i,1} = \frac{1}{4} \max(0, 3) + \frac{3}{4} \max(0, 0) = \frac{3}{4}.$$

If the full output row has three physical columns and cleaning is enabled, it is $[3, \frac{3}{4}, -\infty]$; the compressed output is $[3, \frac{3}{4}]$. A paged call may fetch k_0, k_1 from nonconsecutive physical blocks, but after logical reconstruction it owes these same two scores and the same validity boundary.

The SM100 FP4 representation packs two values per byte. Its query scale has one packed UE8M0 record per token and head, and each KV row has its own packed scale. The exposed FP4 head dimensions are 64 or 128; the FP8 surface also accepts 32. SM100 accepts 8, 16, 32, or 64 query heads, while the pinned

SM90 FP8 implementation accepts 32 or 64 query heads. These are indexer eligibility facts, not generic FP4 attention claims.

Paged decode changes the address calculation. The fused KV cache stores one KV head in blocks. Each token record contains either FP8 bytes followed by an FP32 scale or packed FP4 bytes followed by a packed scale. The block table maps logical blocks to those physical blocks, and the two-dimensional context_lens tensor gives each query row's logical limit. The metadata helper converts those lengths into a $(\text{num_sms} + 1) \times 2$ int32 schedule. The schedule must be rebuilt when the SM budget, context-length state, or indices/varlen mode changes. On SM90 the page size is also a scheduling input. On SM100 it remains an execution-eligibility field, but the metadata generator does not consume it.

The schedule exists because assigning one query row to one SM would balance query count but not work: two rows with very different context lengths contain very different numbers of split-KV segments. The retained SM90 helper instead atomizes the valid work, takes a prefix sum of segment counts, partitions that one-dimensional interval almost evenly, and maps the interval boundaries back to query-token and local-split coordinates [81].

Definition 23.11.2 (Paged-MQA segment schedule) *Let atom a begin at query token t_a , have effective context length ℓ_a , and contain $n_a = \lceil \ell_a / L \rceil$ split-KV segments for split span L . Define $P_a = \sum_{j=0}^a n_j$ and $T = P_{A-1}$. A schedule for S SMs is a sequence of $S + 1$ boundaries whose flattened positions $0 = b_0 \leq b_1 \leq \dots \leq b_S = T$ give SM s the half-open interval $[b_s, b_{s+1})$. Correctness requires those intervals to be disjoint and to cover all T segments; balance requires their sizes to differ by at most one. In indexed variable-length mode, the retained SM90 source pairs adjacent query rows with the same index into one atom, records the first token, and uses the second row's context length [81].*

Algorithm 8: Prefix-sum construction of the retained SM90 paged-MQA schedule.

Input: Query rows with context lengths; optional variable-length indices; split span L ; SM budget S

Output: An $(S + 1) \times 2$ boundary schedule of token starts and local splits

```

1 scan the query rows into atoms  $(t_a, \ell_a)$ , pairing adjacent rows exactly when indexed
  variable-length mode gives them the same index;
2 if there are no atoms then
3   fill all boundaries with the end sentinel and return the schedule;
4 set  $P_{-1} \leftarrow 0$ ;
5 foreach atom  $a$  do
6   set  $n_a \leftarrow \lceil \ell_a / L \rceil$  and  $P_a \leftarrow n_a + P_{a-1}$ ;
7 set  $T \leftarrow P_{A-1}$ ;
8 if  $T = 0$  then
9   fill all boundaries with the end sentinel and return the schedule;
10 set  $q \leftarrow \lfloor T/S \rfloor$ ,  $r \leftarrow T \bmod S$ , and  $p \leftarrow S - r$ ;
11 for  $s \leftarrow 0$  to  $S$  do
12   set  $b_s \leftarrow sq + \max(0, s - p)$ ;
13   if  $b_s = T$  then
14     store the end sentinel;
15   else
16     binary-search the first atom  $a$  with  $P_a > b_s$ , set  $u \leftarrow b_s - P_{a-1}$ , and store  $(t_a, u)$ ;
17 return the boundaries; SM  $s$  consumes the segments between entries  $s$  and  $s + 1$ ;

```

Algorithm 8 has three phases: form atoms, prefix-sum their segment counts, and invert balanced flattened boundaries by binary search. It takes $O(A + S \log A)$ time and $O(A + S)$ metadata space. The retained SM90 scheduler gives the extra r segments to the final r SM intervals: the first $S - r$ intervals have size q , and the rest have size $q + 1$. When $T < S$, repeated zero boundaries safely encode empty early intervals; the nonempty tail intervals still receive one segment each. The schedule must be rebuilt after any input that changes atomization, segment counts, or S .

Example 23.11.2 (A seven-segment paged-MQA schedule) Suppose three atoms have segment counts $(1, 4, 2)$, so their inclusive prefix sums are $(1, 5, 7)$. With $S = 3$, the construction gives $q = 2$, $r = 1$, $p = 2$, and flattened boundaries $(b_0, b_1, b_2, b_3) = (0, 2, 4, 7)$. Table 23.23 shows the inverse mapping. Every segment appears once, and the interval sizes are $(2, 2, 3)$.

Figure 23.13 separates the address path from the score oracle. Dense and paged calls can recover the same logical key k_j through different metadata.

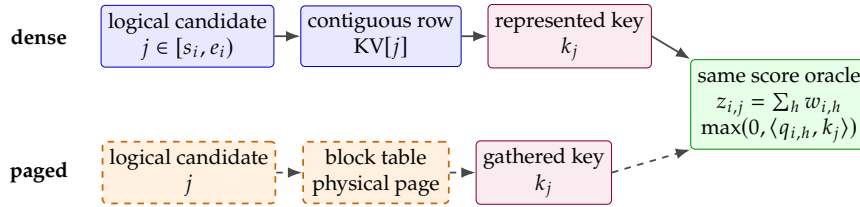


Figure 23.13: Dense and paged MQA kernels use different address metadata to recover the represented logical key k_j , then owe the same weighted-ReLU head reduction. Solid and dashed paths remain distinct without color; the shared oracle prevents a physical page number from replacing the logical candidate coordinate.

Algorithm 9 first derives the logical candidate domain, leaving every unfilled full-output coordinate at $-\infty$. Then dense and paged modes use different address paths to reconstruct the represented key for the same logical j . Finally, both paths apply the weighted-ReLU head reduction and retain the output-coordinate map. This prevents a physical page address or compressed offset from being compared as though it were the logical cache position.

How full-output cleaning is implemented. The cleanup is itself a CUDA algorithm rather than a host-side fill. The recorded `smxx_clean_logits` kernel prepares a shared-memory slab of $-\infty$, partitions output rows across SMs and warps, fills each assigned row's invalid prefix and suffix, and waits for the primary logit kernel through a grid dependency before touching the output. It uses bulk copies for aligned interiors and scalar stores for boundary fragments, so valid logits are preserved while every unfilled full-matrix coordinate becomes exactly $-\infty$ [81]. Compressed-output mode does not owe those physical gaps, and benchmark records keep this cleaning kernel's time separate from score-kernel time.

The repository test generates dense and paged BF16 references, simulates the chosen FP8 or FP4 quantization, and compares the kernel with both. That second

SM	Flat interval	Start boundary	Assigned atom:split pairs
0	$[0, 2)$	$(a_0, 0)$	$(a_0, 0), (a_1, 0)$
1	$[2, 4)$	$(a_1, 1)$	$(a_1, 1), (a_1, 2)$
2	$[4, 7)$	$(a_1, 3)$	$(a_1, 3), (a_2, 0), (a_2, 1)$

Table 23.23: Prefix-sum inversion for segment counts $(1, 4, 2)$. The terminal boundary at 7 is the end sentinel.

Algorithm 9: Shared logical oracle for dense and paged MQA indexer logits.

Input: Quantized queries and KV records with scales; head weights w ; dense or paged mode; interval/context/index metadata; full or compressed output mode

Output: Indexer logits and a logical-to-output coordinate map

```

1 initialize full output coordinates to  $-\infty$ , or initialize empty compressed rows;
2 foreach query row  $i$  do
3   derive the valid logical candidates  $C_i$  from  $[s_i, e_i)$ , or from the declared context lengths and
   optional indices;
4   foreach logical candidate  $j \in C_i$  do
5     if the mode is dense then
6       set the KV address  $a \leftarrow j$ ;
7     else
8       decode the declared index format and resolve  $j$  through the block table to physical
       address  $a$ ;
9     reconstruct each represented  $q_{i,h}$  and  $k_j$  from its payload and scale;
10    set  $z_{i,j} \leftarrow \sum_h w_{i,h} \max(0, \langle q_{i,h}, k_j \rangle)$ ;
11    write  $z_{i,j}$  at logical column  $j$ , or at the recorded local offset when compressed output is
    requested;
12 return logits plus the map from every stored value back to logical  $j$ ;

```

Table 23.24.: The indexer test oracle reconstructs logical candidate positions as well as comparing the dense result tensor.

Mode	Required state	Oracle
Dense full logits	Quantized q and kv , scales, head weights, s_i, e_i , output dtype, cleaning flag	ReLU-weighted head reduction, exact $-\infty$ mask, numerical difference, and repeated bitwise self-consistency.
Dense compressed logits	Same state plus maximum local key-span width	Expand row i into $[s_i, e_i)$, then apply the full-logit oracle.
Paged logits	Quantized query, fused paged KV bytes and scales, context lengths, block table, schedule metadata, maximum context, optional indices	Gather logical pages, enforce per-query limits, compare valid logits and invalid mask, and repeat for self-consistency.
Output cleaning	Logical valid-position mask and output dtype	Every unfilled full-matrix coordinate is $-\infty$; cleaning time is reported separately from score-kernel time.

comparison separates quantization loss from kernel error. It also repeats each kernel twenty times and checks bitwise self-consistency on valid coordinates. The earlier H200 closeout completed all 192 combinations selected by the upstream SM90 FP8 dense-prefill enumerator but did not serialize each difference. A reviewed follow-up adds two dense and two page-64 cases with exact `calc_diff` values under the same metric used in Section 23.10. The metadata obligations in Table 23.24 govern both packets; Table 23.25 records the four new cases.

The full-logit case preserved the logical negative-infinity mask, while the compressed case was compared after restoring its logical interval. The paged oracle compared and hashed only valid logical positions; it did not read or reinterpret unfilled columns. A fifth selector case requested automatic dispatch at page size 32. It selected the reference implementation with reason `deepgemm_sm90_paged_mqa_requires_page_size_64`, made one reference call, and launched no DeepGEMM kernel.

Kernel attribution is independent of the numerical comparison. The direct profiler observed `deep_gemm::sm90_fp8_mqa_logits`, `deep_gemm::sched::sm90_pag`

Case	Recorded SM90 shape	BF16 reference	Simulated quantized reference
Dense full, FP32 logits	$S_Q = 2048, S_{KV} = 8192, H = 32, D = 64$	6.626×10^{-4}	1.033×10^{-6}
Dense compressed, BF16 logits	$S_Q = 2048, S_{KV} = 8192, H = 64, D = 128$, with context-parallel ranges	6.828×10^{-4}	2.383×10^{-6}
Paged boundary, FP32 logits	$B = 8, H = 32, D = 128$, next_n = 1, $P = 64$; lengths 63, 64, 65, 127, 128, 129, 511, 512	4.748×10^{-4}	7.723×10^{-7}
Paged long, BF16 logits	$B = 4, H = 64, D = 128$, next_n = 2, $P = 64$; paired limits from 1023/1024 through 8191/8192	7.547×10^{-4}	2.576×10^{-6}

Table 23.25.: Serialized H200 dense-indexer and paged-MQA numerical differences. Every row passed the 10^{-3} BF16-reference threshold and the dtype-specific simulated-reference threshold, and repeated bitwise-identically twenty times. These are direct-kernel results, not model-backed indexer-serving measurements.

ed_mqa_logits_metadata, and deep_gemm::sm90_fp8_paged_mqa_logits; twelve generated JIT artifacts were identified. These observations cover the recorded current-device paged and serialized-difference cases. DeepSeek-V2-Lite has no V3.2 lightning indexer, and the official 671B V3.2 artifact exceeds four H200s. The model-backed serving route was therefore not measured.

23.12. Summary

DeepGEMM preserves each output relation while changing representation, layout, schedule, and compiled path. Dense and K-grouped surfaces can use $D = C + AB^T$; the checked M-grouped wrappers omit C and write $D = AB^T$. FP8 and FP4 payloads remain attached to scales, accumulation, and conversion. M-grouped work varies valid expert rows; K-grouped GEMM instead varies the reduction extent while retaining one fixed-shape output per group. The head-split epilogue changes only placement by scattering computed columns around caller-owned gaps.

The fast path combines block-scaled partials, staged TMA movement, matrix consumers, persistent tile claims, schedule swizzles, architecture-specific accumulators, and JIT-specialized cubins. SM90 WGMMMA keeps accumulator fragments in registers; SM100 Tensor Memory holds the same logical D across K -slice updates until TCGen05 completion and register extraction. Cold and warm timing is comparable only when shape, layout, artifact, cache state, launch policy, and numerical oracle match.

The grouped-expert output contract becomes an input to Chapter 27, where communication, two expert linears, activation, and combine form a separate systems interval. MQA/indexer dense and paged address paths therefore meet at one logical score oracle. Einsum and HyperConnection prenorm retain their own reduced axes, outputs, and conversion boundaries.

Generated CUDA, PTX, cubin, SASS, and launch records answer different questions. The retained H200 rows establish selected SM90 timings and route attribution under recorded conditions; FP4 MQA performance remains outside that measurement set. Mega MoE owns its separate SM100 evidence record.

23.13. Exercises

Use these exercises to practice carrying the same GEMM claim through shape, layout, dtype, JIT, measurement, and tolerance records.

1. CORE: For $M = 6, K = 8, N = 12$, give the shapes under $D = C + AB^T$ and compute multiply-add and scalar-FLOP counts.
2. CORE: Separate mathematical GEMM validity from DeepGEMM layout, dtype, architecture, and scale eligibility. For head splits (2, 1, 2), map eight columns into the widened store and identify reserved columns and their initializer.
3. APPLIED: Reproduce the chapter's block-scale trace: formula and scale shape, a swapped-scale mutation, two $K = 256$ promotion updates, and packed UE8M0 word. Run the format-neutral oracle in `chapter_deepgemm/` and classify its fixed-payload scale-pairing failure.
4. CORE: For `bnk, bnk -> mn`, identify reductions and output shape. State FP32 C/D aliasing versus BF16 workspace rules; reduce partials 11 and 10, naming the SM90/SM100 mechanisms and legal BF16-conversion point.
5. APPLIED: For M-grouped row counts (5, 4, 3), write all three equations, state that the wrapper has no additive C input, and identify shared versus varying dimensions.
6. CORE: Contrast contiguous and masked grouped layouts and give one useful MoE case for each.
7. APPLIED: For a K-axis grouped contiguous GEMM, reproduce the two-group example and contrast it with M-grouping. For end offsets (3, 9), alignment four, derive valid ranges and the gap; state how both scale-packing paths preserve its invalidity.
8. RESEARCH: Reproduce the compiled-path analysis: deterministic JIT selection, cold/warm timing, wave fill for N -tiles 128 and 112, persistent-slot ownership for the 352-tile plan, and two-CTA multicast. Relate resource constraints, retained artifacts, runtime selection, and timing boundaries.
9. APPLIED: For `tf32_hc_prenorm_gemm`, write D and row-square-sum references. With four splits, give intermediate shapes and both reductions; explain why matching only D fails.
10. APPLIED: Recompute the weighted-ReLU MQA example: valid scores, cleaned and compressed outputs, logical-order-preserving page movement, and aligned-plus-scalar cleanup behavior.
11. RESEARCH: Build the chapter's MQA indexer evidence record and derive the (1, 4, 2), three-SM schedule. State rebuild triggers and the grouped-MoE evidence boundary. Only the SM90 FP8 dense direct-kernel matrix is measured; paged, FP4, and model-backed V3.2 rows remain unmeasured.